

Quantitative Analysis of Temperature-Dependent Brownian Motion via 2D Video Microscopy

Vultaggio Yann, Lucien Lamy, Nassima Sellami-Touil¹ and

Tutor: Dr. Steeve Cronenberger²

¹Faculté des Sciences, Univ. Montpellier, 34095 Montpellier, France

²Laboratoire Charles Coulomb, UMR 5221 CNRS/Univ. Montpellier, 34095 Montpellier, France

Submitted: 09 April 2025

Abstract

English :

Brownian motion remains a fundamental phenomenon in statistical physics, with implications across disciplines from materials science to biology and medicine. In this study, we investigate the temperature dependence of the diffusion coefficient for micron-sized polyester beads in an aqueous solution. Using 2D video microscopy and a particle tracking algorithm, we recorded particle trajectories across a temperature range of 28.56 °C to 46.30 °C. Our results demonstrate a linear correlation between the diffusion coefficient and temperature, consistent with the Stokes-Einstein relation, and a diffusive exponent close to 1 as expected for Brownian motions. We report a measured value for the radius of the beads that closely matches the manufacturer's specifications, validating our experimental approach. These findings not only confirm theoretical predictions but also aim to provide a practical and consistent framework for studying Brownian motion with video-tracking methodologies across diverse thermal datasets. Furthermore, we present our analytical pipeline designed to minimize user-dependent bias, ensuring high reproducibility across varying experimental conditions. Finally, we will discuss the limitations of this user-independent approach.

Français :

Le mouvement brownien demeure un phénomène fondamental de la physique statistique, avec des implications dans de nombreuses disciplines allant de la science des matériaux à la biologie et la médecine. Dans cette étude, nous examinons la dépendance thermique du coefficient de diffusion de billes de polyester micrométriques en solution aqueuse. En utilisant la microscopie vidéo 2D et un algorithme de suivi de particules, nous avons enregistré les trajectoires des particules sur une plage de température allant de 28.56 °C to 46.30 °C. Nos résultats démontrent une corrélation linéaire entre le coefficient de diffusion et la température, en accord avec la relation de Stokes-Einstein, et un coefficient diffusif proche de 1 comme attendu pour un mouvement brownien. Nous rapportons une valeur mesurée pour le rayon des billes qui correspond étroitement à la valeur spécifiée par le fabricant, validant ainsi notre approche expérimentale. Ces conclusions confirment non seulement les prédictions théoriques, mais visent également à fournir un cadre pratique et cohérent pour l'étude du mouvement brownien via des méthodologies de suivi vidéo sur des jeux de données à températures variées. De plus, nous présentons notre chaîne de traitement analytique conçue pour minimiser les biais dépendants de l'utilisateur, garantissant ainsi une reproductibilité élevée sous diverses conditions expérimentales. Finalement, nous discutons les limitations de cette approche indépendante de l'utilisateur.

Keywords: Brownian motion, diffusion coefficient, Stokes–Einstein relation, video microscopy, particle tracking, mean squared displacement, polyester beads, temperature dependence

Contents

1	Experimental Apparatus	3
1.1	Sample, software and camera settings	3
1.2	Brownian motion's ideal conditions	3
1.3	Experimental protocol for measurement	3
2	Tracking Methodology	4
2.1	Preprocessing	4
2.2	Feature Localization	4
2.3	Trajectory Linking	5
2.4	Stubs Filtering	7
2.5	Trajectory Quality Control	7
2.6	Static Particles Filtering	8
2.7	Encumbered Trajectories Filtering	8
2.8	Flow Trajectories Filtering	9
2.9	Drift Correction	12
2.10	Extracting the Diffusion Coefficient	13
3	Results	15
4	Discussion	17
5	Conclusion	21
A	: Derivation of the maximum search range radius formula	24
B	: Derivation of the stretching ratio	24
C	: Summary of measurement results for every dataset	25
D	: List of Equipment	27

Introduction

When this physical phenomenon was first discovered, ‘Brownian motion’ was the name given to the movement of pollen particles suspended in water, as described by Robert Brown in 1828. Building on this observation, Sutherland (1904) and Einstein (1905) independently derived the Stokes-Einstein relation :

$$D = \frac{k_B T}{6\pi\eta r} \quad (1)$$

This relation predicts that the diffusion coefficient D of a spherical particle is linearly proportional to temperature T and inversely proportional to both the fluid viscosity η and the particle radius r . Beyond its theoretical significance, Brownian motion has since found broad applications across disciplines, from the characterization of colloidal suspensions in materials science, to intracellular transport in biology, and even stochastic modeling in finance.

Despite this maturity, experimental verification of the Stokes-Einstein relation remains a valuable exercise, particularly as video microscopy and particle tracking algorithms have made it increasingly accessible. However, these measurements are sensitive to the arbitrary parameters of tracking algorithms. These introduce systematic bias and limits reproducibility across different researchers and datasets.

In this work, we use 2D video microscopy to track micron-sized polyester beads in aqueous solution across a controlled temperature range, with the dual aim of confirming the diffusive nature of the motion and quantitatively testing the Stokes-Einstein relation. A particle-tracking algorithm extracts trajectories from the recorded images, from which we compute the Mean Squared Displacement (MSD), the average squared distance travelled by a particle over a time interval τ :

$$\text{MSD}(\tau) = \langle \|\mathbf{r}(t + \tau) - \mathbf{r}(t)\|^2 \rangle \quad (2)$$

For a purely diffusive process, the MSD grows linearly with lag time, $\text{MSD}(\tau) = 2dD\tau$, where d is the number of spatial dimensions, allowing D to be extracted from the slope. Repeating this measurement across a range of temperatures then provides a direct quantitative test of the Stokes-Einstein relation. Fitting the MSD to a power-law form $\text{MSD}(\tau) \propto \tau^\alpha$ also allows us to confirm that the motion is indeed diffusive, as indicated by an exponent α close to 1.

A particular emphasis is placed on building an analytical pipeline that minimizes arbitrary and subjective user input, in order to ensure consistent and reproducible results.

Section 1, describes the experimental setup, from the sample conception to the measurement of temperature and recording of our datasets. Section 2 goes through the whole tracking methodology, including the different filters applied to trajectories to ensure a bias-free analysis, how every parameter is fixed and finally how are results extracted from it. Section 3 presents our results, and Section 4 discusses them in the context of the Stokes-Einstein relation and the limitations of our approach.

1 Experimental Apparatus

1.1 Sample, software and camera settings

For our observations, we use colloidal polyester particles with a diameter of $D = 5.93 \mu\text{m}$, as specified by the manufacturer. We then prepare a dilution in water with a concentration of 10^{-4} using [pipettes](#). All our observations are carried out using this last solution and an Infinity 1 [camera](#) mounted on a [microscope](#) and fitted with a $10\times$ objective. We first check the image captured by the camera via the preview function in the Infinity-Capture software; brightness and focus are adjusted directly via the microscope. We determine the size of a pixel in the image using a graduated microscope slide; we find that $1\text{pixel} = 0.516 \mu\text{m}$, from which we note $\rho_{\mu\text{m}}$, our pixel-to-micron conversion factor. Finally, we set the various parameters in the [InfinityCapture](#) software that are relevant for our recordings. The fixed settings are resolution: 2048×1536 and exposure: 0.067s ; the gain depends on ambient light but is generally low, and the capture time is determined according to our needs and will be specified in detail each time (The goal was to have recordings of identical lengths, but due to software bugs, this was not always possible.) Finally, we disable the preview during recording to avoid potential software-related issues and allow the program to focus its computational power on a proper recording instead.

1.2 Brownian motion’s ideal conditions

First of all, we need to ensure that our particles are indeed undergoing Brownian motion. To achieve this, the particles have to be thoroughly suspended in the fluid and therefore subjected as little as possible to external mechanical forces. The standard method of microscopic observation, which involves placing our solution between two glass slides, is not feasible because both slides exert pressure on the fluid, effectively impairing our particle’s movement. Using a tank would not be ideal either; indeed, if the vertically allowed space were too large, the particles would disappear during long recordings due to gravity pulling them down. This is why we decided to proceed as follows: on a glass slide, we place four pieces of adhesive tape to form a small container, slightly smaller than the upper cover slip and of very small volume; this allows us to avoid the pressure caused by the conventional method and sufficiently reduce the effect of gravity to avoid disrupting a complete recording.

1.3 Experimental protocol for measurement

The main challenge to ensure recording of a sample subjected to heating is that our system has to maintain a uniform temperature to prevent any convective movement, which would hinder long-term recordings. We therefore use two metal plates, each with a hole in the center to allow observation, between which we place the glass slide, and above which we position a [resistor](#). It is also important to fit plastic pads to the metal base plate

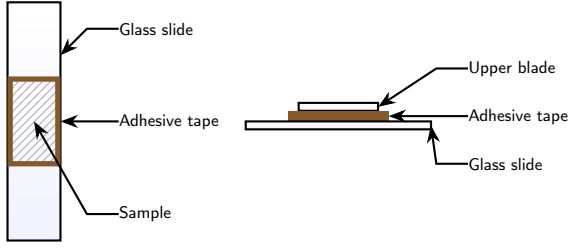


Figure 1: View from above / view from the side, of the sample preparation for the verification of Brownian motion at ambient temperature. The sample is placed between two glass slides, but they are separated by a layer of adhesive tape that creates a small container for the solution, allowing the particles to move freely without being subjected to pressure from the slides.

to prevent damage to the microscope and heat transfer towards the base of the microscope that could result in an additional loss of heat in our system. Sometimes a bubble may appear during heating; if it is not too large, we can ignore it, otherwise we add some of our solution to fill it and wait until the temperature is uniform again. To effectively heat our sample, we use a thermal resistance connected to a [generator](#). Temperature measurements are done by monitoring the resistance using a [multimeter](#) (Fig. 3). Temperature monitoring is essential to properly analyze our data. Care must also be taken to ensure that the temperature remains consistent throughout the recording. We therefore wait for the resistance value to stabilize, and add an extra minute for the sample to reach thermal equilibrium. In case of temperature variation, we note the value of the resistance at the start and end of the recording. After that we need to determine the relationship between the temperature and the resistance value. To do this, we refer to experimental data of a calibration by the OECS (Optique des États Collectifs et du Spin) conducted using the absorption spectra of a rubidium vapor, from which we derived, using `Python`, a calibration curve giving us the relationship between temperature and resistance of this specific thermal resistance (Fig. 4):

$$T \approx 485.4990R^{-0.0876} + 83.1835 \quad (3)$$

Also, to prevent any discrepancy in our data due to an unexpected uncertainty on this thermal calibration curve, we measure the room temperature using both a thermometer and a resistance meter, from which we deduced the effective offset in temperature.

$$\begin{aligned} T_{\text{thermo}} &\approx 25.00 \pm 0.05^\circ\text{C} \\ T_{\text{res}} &\approx 26.75 \pm 0.15^\circ\text{C} \end{aligned} \quad (4)$$

We observe a lower temperature on the thermometer, which gives us an effective offset of $\Delta T = 1.75 \pm 0.05^\circ\text{C}$

2 Tracking Methodology

We perform the tracking using the Crocker-Grier centroid-finding algorithm [1], through the trackpy pack-

age [2] that we implement in our own `Python` environment. These algorithms represent the established standard in the field and are widely utilized across the microrheology community. While trackpy offers a standardized pipeline for feature localization and linking, we implement additional filtering layers and a calibration framework to ensure the integrity of the resulting dataset beyond the basic detection thresholds. This multi-step validation process is designed to mitigate the effects of residual optical noise and thermal gradients, resulting in a high-fidelity dataset that remains unbiased even under the constraints of a low-cost experimental environment. To ensure full reproducibility and transparency in our trajectory analysis, we present our integrated tracking workflow. This pipeline encompasses the foundational feature-localisation and linking stages established by the Crocker-Grier algorithm, seamlessly interfaced with our custom diagnostic and filtering layers. By detailing the complete sequence - from raw pre-processing to our specialized GMM-based static-particle discrimination - we provide a comprehensive framework for 'bias-free' Brownian motion analysis in variable-noise environments. All these filters and steps will be detailed in the following sections.

2.1 Preprocessing

Starting from raw .avi video files, we use ImageJ [3] to extract individual frames converted into .tif format and apply a grayscale conversion to ensure a good signal-to-noise ratio and reduce the computational load for the subsequent tracking steps. Using video metadata and frame count, we provide the program with the necessary frame rate information to ensure accurate temporal resolution in the resulting trajectories. We also provide the program with the pixel-to-micron conversion factor $\rho_{\mu\text{m}}$ calculated based on a reference image of a micrometer scale seen through the microscope, which we then compare to the on-screen resolution of the camera.

2.2 Feature Localization

To locate features in each frame, trackpy requires two key parameters: the estimated optical diameter of the particles, and a minimum mass threshold m_{min} , to filter out noise, where mass here is the name given to the integrated brightness of the detected feature.

To determine the optimal size, we follow the standard subpixel bias localization procedure detailed in trackpy guidelines. Through trial runs with varying diameter values, we look at the resulting distribution of detected features until we find the value that minimizes the bias in the localization of the particles, as evidenced by a symmetric distribution of detected positions around the expected particle centers (Fig. 5b). As observed in Fig. 5a, a diameter value that is too low leads to a significant "pixel-locking" effect, where detected positions are biased towards integer pixel values, resulting in peaks at 0.0 and 1.0 in the sub-pixel distribution. By contrast, the optimized diameter value of 13 pixels yields a flat plateau in the sub-pixel

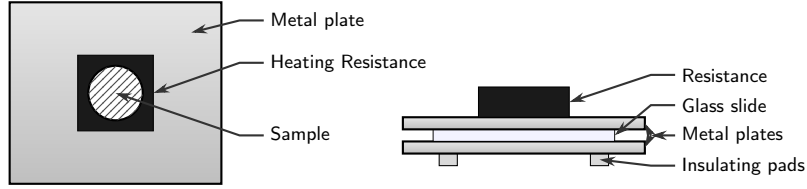


Figure 2: View from above / view from the side, of the heating mechanism used to control the temperature increase of the solution. The sample is placed between two metal plates, which are heated by a resistance. The glass slide is placed on top of the bottom plate, and the upper plate is placed on top of the glass slide, with a hole in the center to allow observation. Insulating pads are placed between the bottom plate and the microscope to prevent damage and heat loss due to transfer towards the microscope.

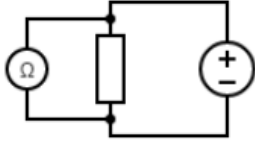


Figure 3: Circuit diagram

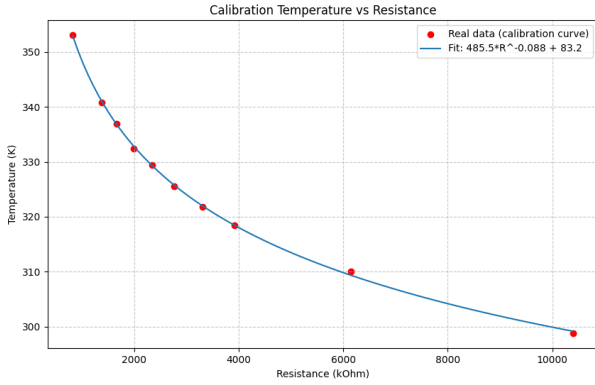


Figure 4: Calibration curve of our thermal resistance, giving us a relationship between the resistance measured and the actual temperature of the resistance.

distribution, indicating unbiased localization of particle centers across the pixel grid. The chosen diameter of $d_{\text{pix}} = 13\text{px}$ is then validated by comparing it to the physical dimensions of the polyester beads. Given our calibration factor of $M \approx 0.516\mu\text{m px}^{-1}$, we get $d = 6.71\mu\text{m}$, which is a slightly larger value than the manufacturer’s specified diameter of $5.93\mu\text{m}$. It is an expected offset, consistent with the optical blurring and diffraction effects in our imaging system that tend to make objects appear larger than their actual size. To fix m_{min} we use a similar approach, by looking at the distribution of detected features across a range of mass values, along with a visual inspection of the resulting selected features. Since this is one of the first filters applied to the raw data, we choose to manually set it at a strict value, to ensure a high-quality dataset as our starting point for the subsequent steps. This choice however requires a sufficient concentration of particles in the field of view to ensure a good statistical sample.

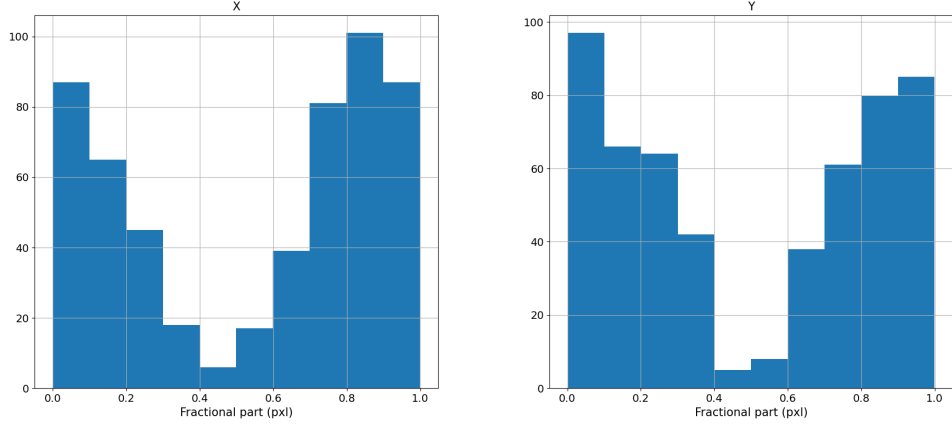
This selection is meant not only to filter out noise, but also particles that are not fully in focus, since they may be part of a population affected by the bottom floor of the sample, and thus not representative of the bulk behavior of the system as it has been shown extensively [4, 5]. The distribution of mass values for the detected features typically follows a Gaussian distribution, with a tail of low-mass features that are likely to correspond to those out-of-focus particles. By setting m_{min} at the base of the Gaussian distribution, we ensure that we are selecting the majority of the in-focus particles while excluding those that are likely to be artifacts, as shown in Fig. 6b. Setting the threshold too low, as shown in Fig. 6a, results in a significant number of low-mass features being included in the dataset, which can introduce noise and bias into the analysis, reducing our effective measurement of D . As a final check, we also look at the resulting size and mass distributions for the detected features to ensure that the population captured by those first parameters is coherent (i.e., homogeneous in size and mass, which is expected from a population of identical particles), as seen in Fig. 7.

By default both of these tests are implemented on the first frame of the video, but they can be applied to any frame or set of frames to ensure that the parameters are consistent across the whole video, and not just for a specific time point.

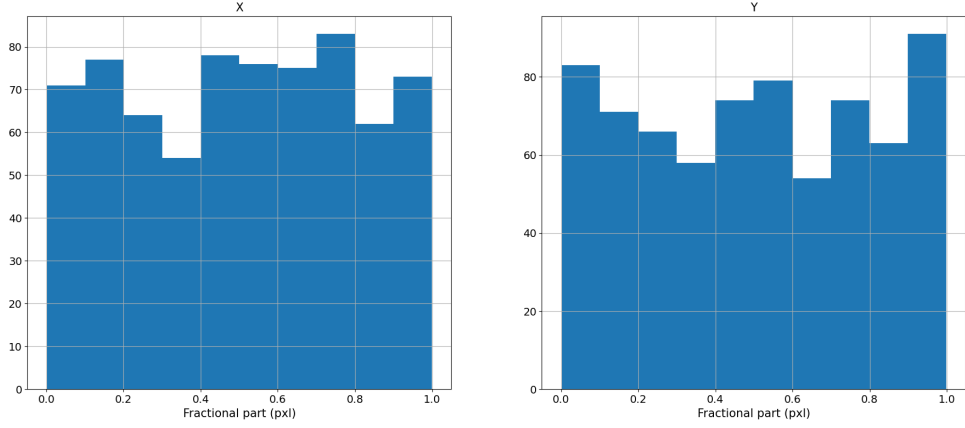
2.3 Trajectory Linking

Trackpy also implements a linking algorithm to connect detected features across frames into trajectories. This process relies on a search range parameter r_s that defines the radius around a particle in frame t , within which the algorithm will search for the same particle in frame $t+1$, and a memory parameter t_{mem} that allows for temporary disappearance of particles (e.g., due to occlusion or noise) without breaking the trajectory (i.e., after a particle disappears, the algorithm will still continue to look for it for t_{mem} subsequent frames).

To set r_s , we use the expected theoretical diffusion coefficient of our system, which can be estimated using the Stokes-Einstein relation. We derive T from the resistance measurement R of our heating system through Eq. (3), where the radius r used for computation is the one provided by the manufacturer, and the water’s viscosity $\eta(T)$ is given by the correlation provided by the



(a) Sub-pixel bias with $d = 7$. Significant "pixel-locking" is observed as peaks at 0.0 and 1.0.



(b) Optimized distribution with $d = 13$. The flat plateau indicates unbiased sub-pixel localization.

Figure 5: Comparison of sub-pixel fractional position distributions. (a) Insufficient diameter leads to systematic bias toward integer pixel values. (b) Correct parameters ensure a uniform distribution.

International Association for the Properties of Water and Steam (IAPWS). Specifically, we implemented the temperature-dependent viscosity formulation [6] within our Python analysis pipeline. Knowing the theoretical estimate of D , we can then calculate the expected mean squared displacement (MSD) for a given lag time interval τ using the relation :

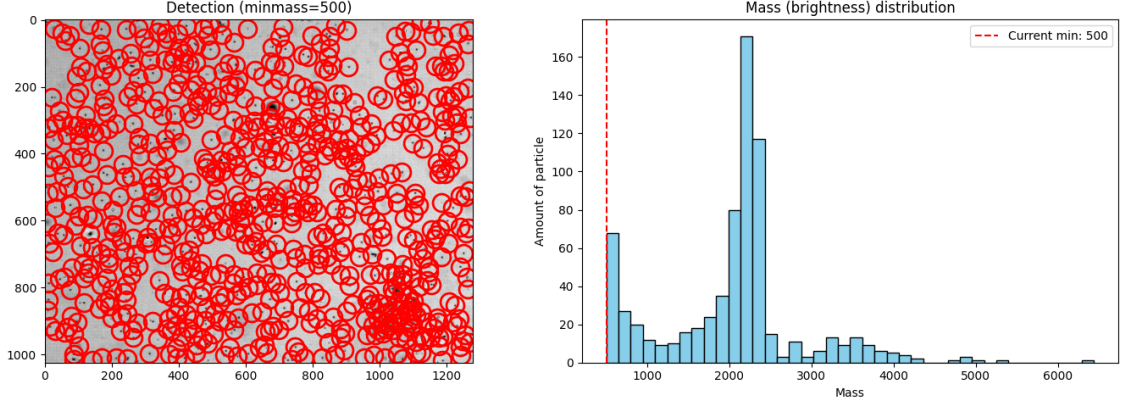
$$\langle \Delta r^2(\tau) \rangle = 4D\tau \quad (5)$$

for 2D Brownian motion. We then introduce a new parameter P_{miss} that sets the probability of a particle moving beyond the search range. We can then compute the maximum r_s as follows :

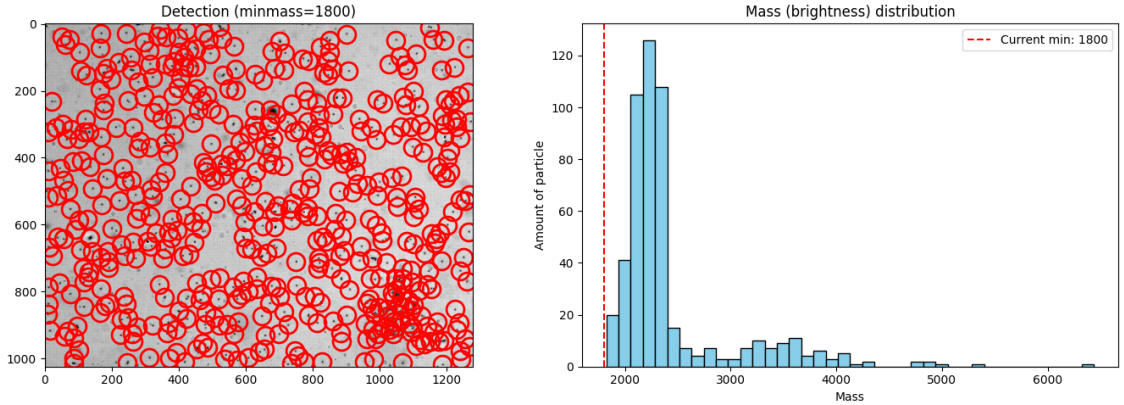
$$r_s = \frac{1}{\rho_{\mu m}} \sqrt{-4D\Delta t \ln(P_{\text{miss}})} \quad (6)$$

A detailed derivation of this relation is provided in the appendix (Section A), the only difference is here we need the value in pixels, hence the division by the calibration factor $\rho_{\mu m}$. From this relation, we gather that

a lower value of P_{miss} results in a larger search range (missing fewer particles), which allows for more flexibility in linking trajectories but also increases the risk of incorrect associations, especially in cases of high particle density or noise. Conversely, a higher value of P_{miss} results in a smaller search range, which can reduce the likelihood of incorrect associations but may also lead to fragmented trajectories if particles move more than the defined range between frames. To fix a higher bound of P_{miss} , we also compute the area per particle of our system, using the density of particles in the field of view, and set P_{miss} to a value that allows for a search range that is smaller than the average inter-particle distance, to minimize the risk of incorrect associations. In our case, we found that we could go down to $P_{\text{miss}} = 0.0001$, which gives $r_s = 1$ pixel. The calculated inter-particle distance being $r_{\text{inter}} \approx 13$ pixels, we do not expect incorrect associations, resulting in a safe search range that includes the vast majority of expected displacements. We do note that this approach requires a test run since



(a) Insufficient mass threshold with $MINMASS = 500$. The distribution includes a significant number of low-mass features, likely corresponding to out-of-focus particles which we can see as a second and lower gaussian distribution at the left of the main distribution.



(b) Optimized distribution with $MINMASS = 1800$. We target the base of the gaussian distribution of our particle population.

Figure 6: Comparison of mass distributions, and visual inspection of the selected features. (a) Insufficient mass threshold leads to inclusion of noise or out-of-focus particles. (b) Optimized threshold ensures a clean selection of most of the particles.

the density we use is the density of the particles that survived all our filters, which is not known beforehand, but it allows us to set the search range to a well-defined value, based on the expected physical behavior of the system and the actual density of particles in our dataset, rather than relying on arbitrary values or too many trial-and-error adjustments.

Neither trackpy guidelines nor Crocker and Grier's methods include a specific rule on how to set the t_{mem} parameter, but a generic rule of thumb is to set it reasonably low to avoid corrupting the dataset with wrongly linked trajectories. Since we do not suffer from significant particle disappearance issues, and we want to minimize the risk of incorrect associations, we set $t_{\text{mem}} = 2$ frames as a precaution. However it seems that setting it to 1 or even 0 does not significantly affect the resulting trajectories, which suggests that our dataset is of good quality and doesn't suffer from significant occlusion or focus issues that would cause particles to disappear temporarily.

2.4 Stubs Filtering

Following the linking process, we apply a "stubs" filter to remove short-lived trajectories that do not provide sufficient data for statistical analysis. This stage eliminates transient artifacts such as fast-moving dust or noise spikes, that bypass initial mass filters but fail to form coherent paths. We use a minimum frame threshold n_{min} which is the minimum number of frames a trajectory must be visible to be considered by the algorithm. According to modern microrheology tracking methods [7], it is advised to set n_{min} so that trajectories achieve a minimum temporal coverage at least two orders of magnitude higher than the frame interval Δt_{frame} . Considering $\Delta t_{\text{frame}} \approx 0.0672$ s, we get $n_{\text{min}} = 15$.

2.5 Trajectory Quality Control

The next filter implemented is a three-parameter filter based on mean values of the particle's mass, size and eccentricity across the trajectory to ensure that we are keeping trajectories that correspond to well-defined particles that are consistently in focus and not affected by

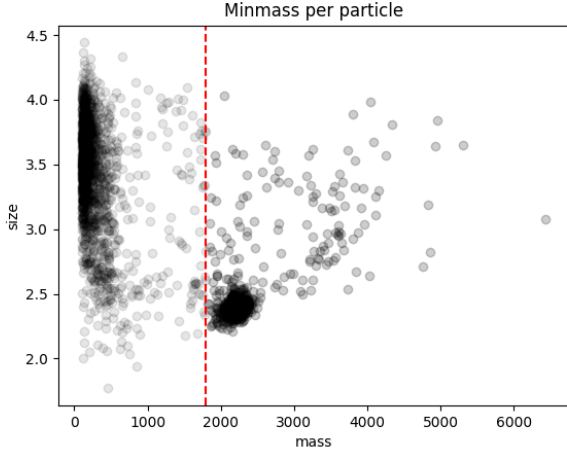


Figure 7: Size and mass distributions for the detected features, used as a visual check to ensure we capture a sufficient number of particles, and they represent a relatively homogeneous population. We can see a relatively narrow distribution of mass and size values next to our minimum threshold (red-dotted line). Near 0 mass, we can see a tail of low-mass features which are likely to correspond to visual artifacts, not actual particles.

significant noise or artifacts across the span of the trajectory. The specific thresholds for each parameter are determined based on the distribution of values across the trajectories, as well as visual inspection of the trajectories that are kept by the filter (Fig. 8 and Fig. 9). Both mass and size thresholds have a lower and an upper bound to ensure we target our intended population of particles. Eccentricity only has a maximum threshold, since an eccentricity of 0 corresponds to a perfect circle, which would be the ideal shape of our particles, while higher values correspond to more elongated shapes that are likely to be artifacts or aggregates.

2.6 Static Particles Filtering

Now that we have a set of trajectories that are well-defined and mostly free of noise and artifacts, we can apply more specific filters to ensure that we are targeting the population of particles that are actually undergoing Brownian motion. As discussed previously in Section 1, one of the main issues we have to deal with is the presence of static or near-static particles that are stuck to one of the glass slide. A common practice in the field of microrheology is to measure the standard deviation (STD) of the displacement of fixed particles (a reference measurement can be used, usually particles glued to the surface) to determine the localization error of the system [8]. However, since we have worked over several weeks, and those reference measurements should be recorded in the same conditions as diffusing particles, we decided to try to use the static particles stuck at the bottom of the sample as our reference population, meaning we can compute this static threshold directly from our dataset. To isolate this static population from our actual Brownian population, we compute the mean positional

standard deviation of each trajectory, the average of the standard deviations of the x and y coordinates across all frames, which we denote $\bar{\sigma}_{pos}$. This quantity is small for immobile particles and should be larger for genuinely diffusing ones. We then fit a two-component Gaussian Mixture Model (GMM), which is an algorithm that tries to identify clustered Gaussian populations with different means and weights. We apply the GMM to the log-space distribution of $\bar{\sigma}_{pos}$, telling the algorithm to look for 2 populations only. We make use of the log-space rather than linear space to account for the heavy left tail of the static population that is usually much larger in linear space, making it hard to separate both populations (Fig. 11). Working in log-space compresses this tail and usually renders both populations approximately Gaussian, allowing the GMM to separate them reliably. The algorithm then locates the Bayes-optimal threshold (i.e., the point where the posterior probability for a particle with a known STD to belong to either component is equal). This crossing point minimizes the total misclassification probability and is found numerically without our intervention. Converting back to linear space gives the final threshold $\hat{\epsilon}_{static}$. We additionally extract the posterior probability of each trajectory belonging to the Brownian component, $p(\text{Brownian})$, and retain only trajectories with $p(\text{Brownian}) \geq P_{min}$, which we set at 0.8. This soft probabilistic classification is more appropriate than a hard threshold when the two populations overlap, a situation that arises particularly at lower temperatures where slower Brownian motion is harder to distinguish from near-static behavior, or at very high temperature, when the amount of static particles is much smaller than the Brownian population, and the GMM struggles to separate both of them properly. As a final self-consistency check, we can later on compare $\hat{\epsilon}_{static}$ to the localization noise σ extracted independently from the ensemble MSD fit (Section 2.10); since both quantities should approximately measure the same physical noise floor through independent routes, good agreement between them provides confidence in both the static filter and the MSD fitting procedure. We do not expect perfect agreement since this filter is meant to capture all static and near-static particles, which can include particles with a slightly higher $\bar{\sigma}_{pos}$ than the actual localization noise, but we can expect them to be of the same order of magnitude.

2.7 Encumbered Trajectories Filtering

Static particles are not the only source of bias that tends to drag down the measured diffusion coefficient; we also have to deal with particles that are not fully static, but are still encumbered by interactions with the bottom floor of the sample, which can lead to sub-diffusive behavior [4][5]. The diffusive behavior of a particle can be characterised by the anomalous exponent α from the power law relation :

$$\langle \Delta r^2(\tau) \rangle = 4D\tau^\alpha \quad (7)$$

We can expect particles that are significantly affected by the bottom floor to exhibit a sub-diffusive behavior

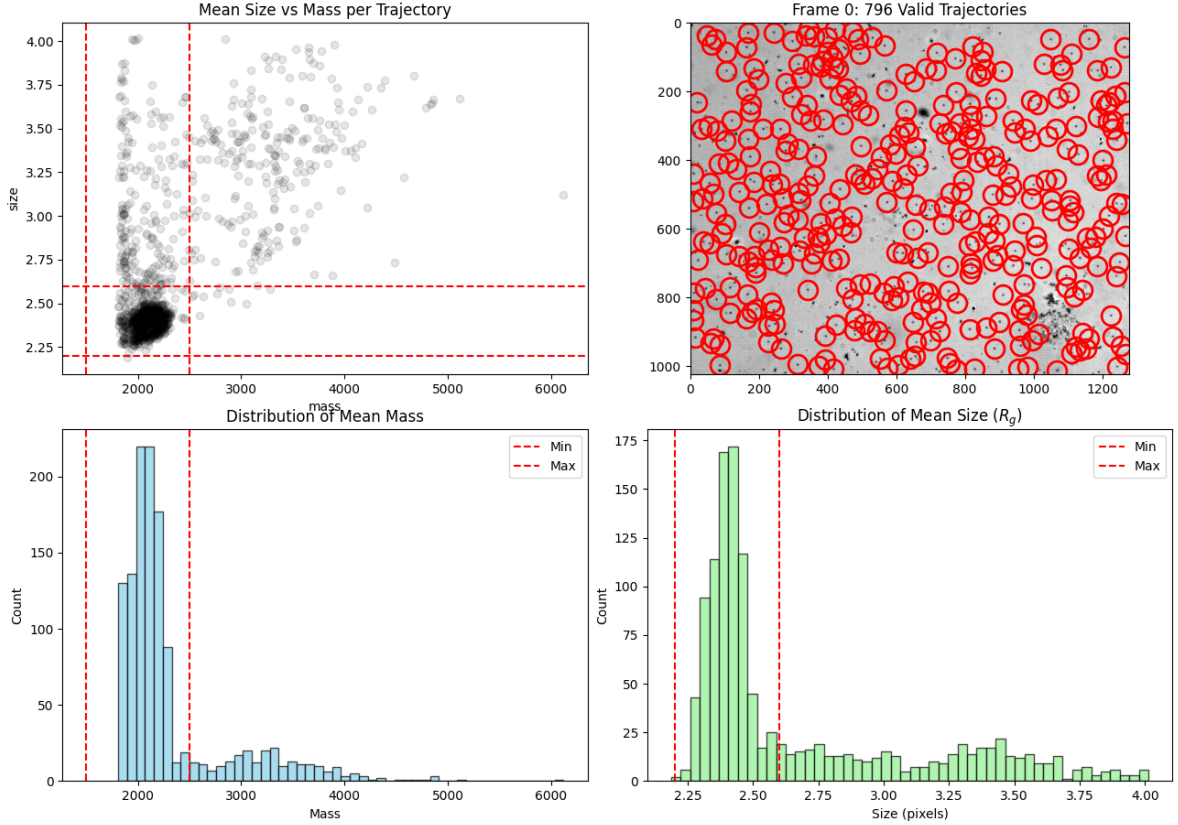


Figure 8: Top left : Mean mass-size distribution for the trajectories, with the thresholds indicated by red-dotted lines. Top right : Visual check on frame 0 of our selected particles circled in red. Bottom left : Histogram of mean mass values across trajectories, with thresholds indicated by red-dotted lines. We can see a proper gaussian distribution, which is our targeted population, and a tail of higher-mass particles which likely correspond to aggregates or dust. Bottom right : Histogram of mean size values across trajectories, with thresholds indicated by red-dotted lines. Once again we see a higher-size tail which likely correspond to aggregates and dusts again.

with $\alpha < 1$. On the other hand, super-diffusive behavior shows an anomalous exponent $\alpha > 1$. For normal Brownian motion, we expect α to be close to 1, which simplifies Eq. (7) to Eq. (5). To filter out those encumbered trajectories, we implement a filter based on α , which we obtain from fitting the MSD curve of each trajectory to a power law. We set a minimum threshold for α at 0.75, in accordance with modern literature [9][10]. Once again, we can use a visual check to compare trajectories that are kept and discarded by this filter as shown in Fig. 10.

2.8 Flow Trajectories Filtering

In addition to static and encumbered particles, we also have to deal with flow-like trajectories that are affected by residual thermal gradients in the sample, which can lead to super-diffusive behavior and an overestimation of the diffusion coefficient. To filter out those trajectories, we implement a filter based on the stretching of the trajectories, which is defined as the ratio of the end-to-end distance of the trajectory to the total length of the trajectory. For a Brownian trajectory, we expect this stretching to be relatively low, since the particle is expected to move in a random walk pattern. However, for flow-like trajectories, we expect the stretching to be much higher, since the particle is expected to move in a

more directed manner. We can calculate the expected value of the stretching for a Brownian trajectory of N steps using the following relation :

$$\langle S \rangle = \frac{\langle R \rangle}{\langle L \rangle} = \sqrt{\frac{\pi}{2N}} \quad (8)$$

This formula is derived in detail in the appendix (Section B). Then, comparing the actual stretching of each trajectory to the expected value, using the number of frames as N , we can set a threshold. To avoid setting an abrupt threshold that would lead to the exclusion of valid Brownian trajectories, we introduce a stretching factor $k_{\text{stretch}} > 1$ that allows for a multiple of the expected stretching to be set as a threshold, giving us the following relation for the stretching threshold :

$$S_t = k_s \sqrt{\frac{\pi}{2N}} \quad (9)$$

Once again a visual check is used to compare trajectories that are kept and discarded by this filter, ensuring that we do not exclude valid Brownian trajectories that may have a higher stretching due to random fluctuations, while still effectively filtering out flow-like trajectories that have a significantly higher stretching, as seen in Fig. 12. As a tool used in our early results, when we were struggling with significant thermal gradients, we

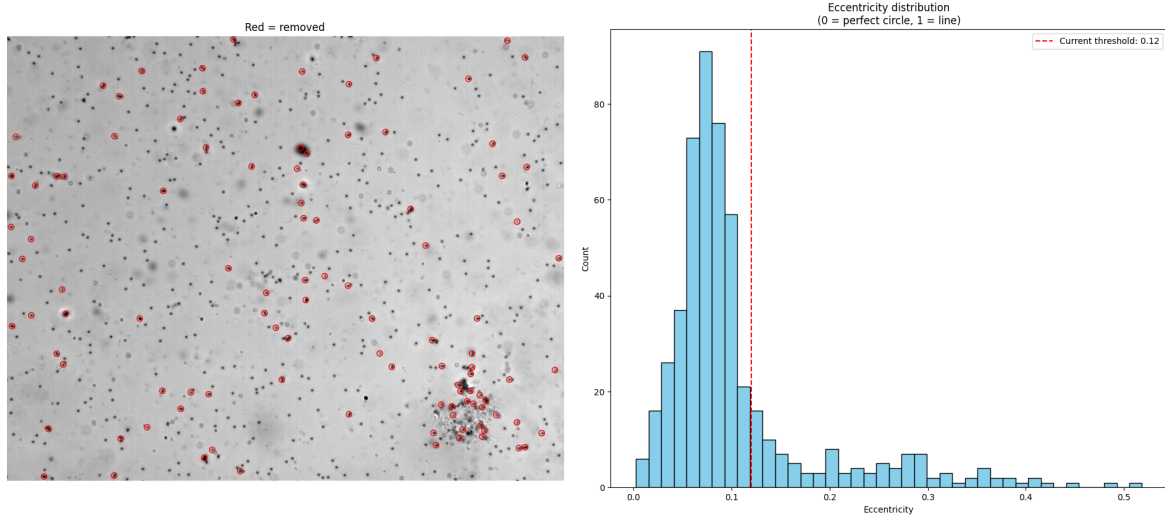


Figure 9: Visual check and histogram of mean eccentricity values across trajectories, with the maximum threshold indicated by a red-dotted line. The distribution is heavily skewed towards low eccentricity values, consistent with spherical particles. Discarded particles (circled in red) are identified as non-spherical artifacts.

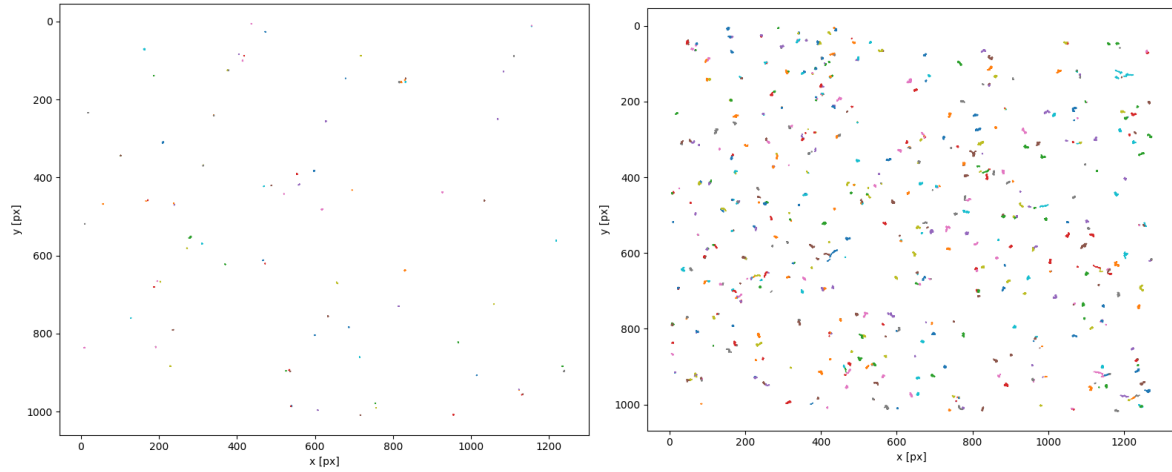


Figure 10: Visual comparison of trajectories based on the anomalous exponent α . Trajectories retained by the filter (right) exhibit characteristic Brownian "drunkard's walk" patterns, while discarded trajectories (left) show restricted, sub-diffusive behavior ($\alpha < 0.75$).

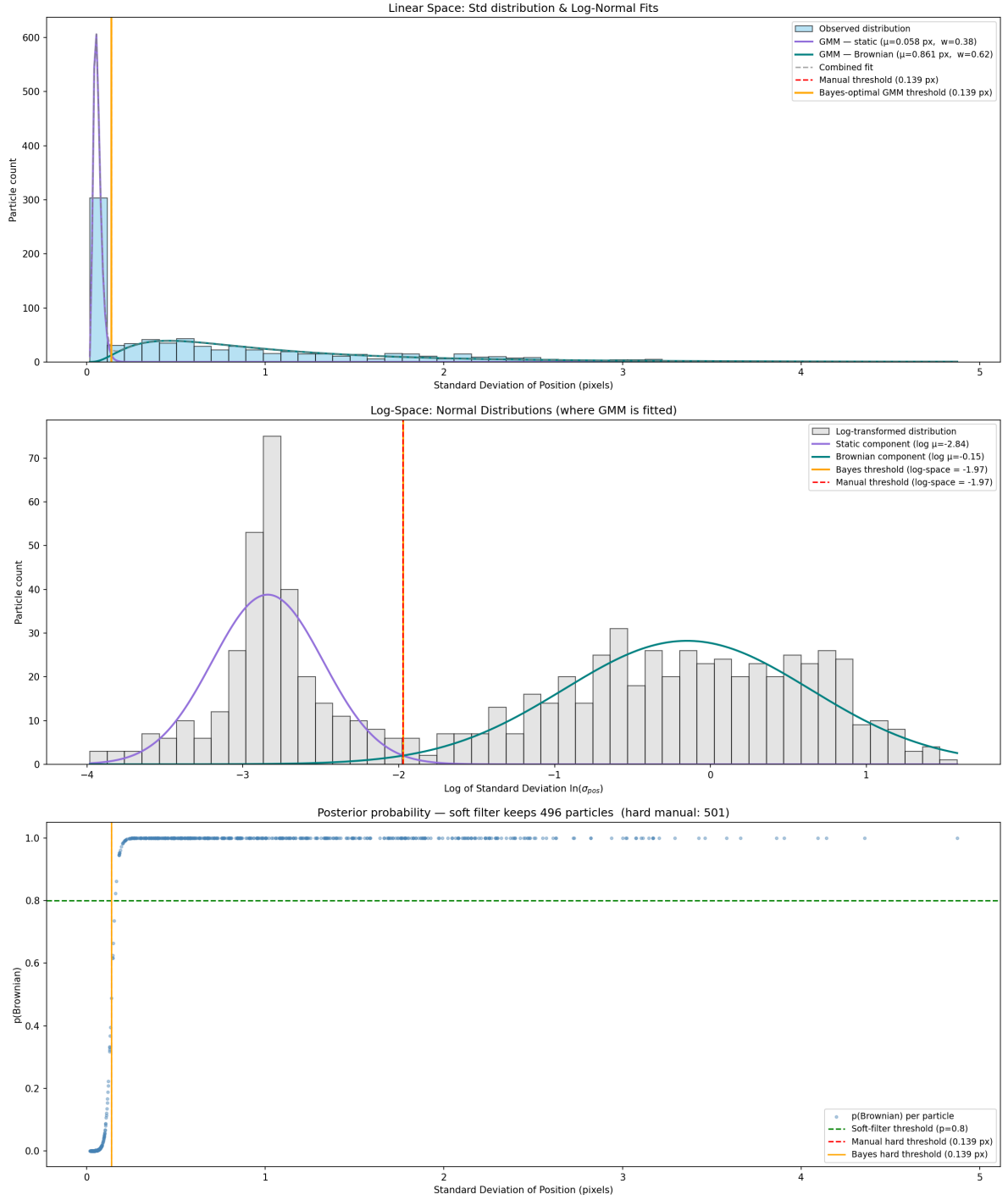


Figure 11: Top : GMM fit of the linear space for comparison, showing the static population as a heavy peak at the left, and the Brownian population as a small plateau right next to it. The GMM struggles to separate both populations in this space, since there is no clear separation between them, and they are not well approximated by Gaussian distributions. Mid : GMM fit of the log-space, showing a good separation between the static population (left) and the Brownian population (right), both well approximated by Gaussian distributions. The yellow line is the current threshold manually set by the user, the red-dotted line shows the Bayes-optimal threshold identified by the GMM. A test run is required to adjust the manual parameter to the Bayes-optimal one. Bottom : A plot of the standard deviation of position (in pixels) of every trajectory as a function of the posterior probability of belonging to the Brownian population. The horizontal green-dotted line indicates the minimum posterior probability threshold P_{min} while the red-dotted and yellow one are the user set and GMM-identified thresholds respectively. We can see that the GMM-identified threshold is consistent with the distribution of trajectories.

also implemented a size-mass distribution check to identify if the flow-like trajectories correspond to a specific population of particles (Fig. 13), we did not find anything conclusive, but this tool can be useful in future experiments to identify if the flow-like trajectories are associated with specific particles.

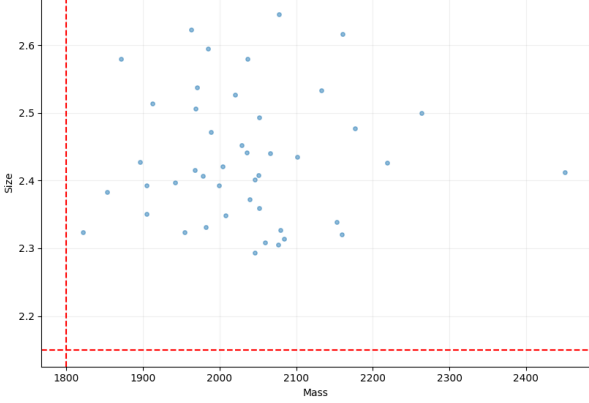


Figure 13: This size-mass distribution check can be used to identify if the flow-like trajectories correspond to a specific population of particles. In this case, we don't see any clear correlation between the stretching and the size or mass of the particles, which suggests that the flow-like behavior is not associated with a specific population of particles. However, this tool can be useful in future experiments to identify if there are specific particles that are more affected by thermal gradients or other sources of bias.

2.9 Drift Correction

Now that we have a set of trajectories cleaned from static, encumbered and flow-like trajectories, we apply a drift correction to ensure that we are measuring the diffusion coefficient of the particles rather than any residual

drift in the system. We use the `subtract_drift` function implemented in `trackpy`, which computes the average displacement of all particles at each time point and subtracts this average from the individual displacements of each particle. This method assumes that the drift affects all particles in a similar way, which is a reasonable assumption in our case since we now have a filtered set of trajectories free of any significant bias. We can also use a visual check to compare the trajectories before and after drift correction to ensure that the correction is effective and does not introduce any new bias into the dataset, as shown in Fig. 15. We can also visualise the average displacement that has been identified as drift in Fig. 14; this is especially useful when struggling with a global drift due to significant thermal gradients or the presence of bubbles in the sample, to ensure that the identified drift is consistent with the trajectories seen in the video.

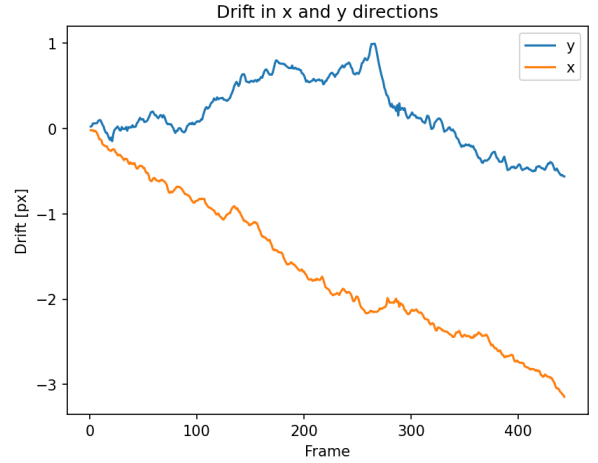


Figure 14: Drift identified in the system by the `subtract_drift` function in `trackpy`. In this example, we can see that the drift is extremely low, moving only a few pixels over the course of the video.

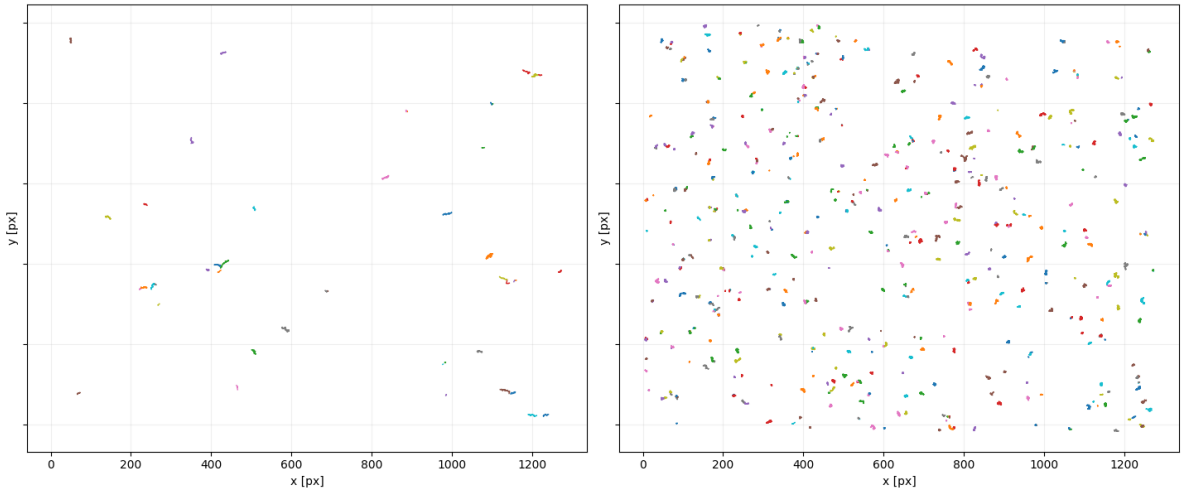


Figure 12: Visual check of stretching trajectories. On the left are shown the trajectories discarded by the filter, on the right are shown the remaining trajectories. Some of the discarded trajectories show a significantly higher stretching, consistent with flow-like behavior. Some of the discarded trajectories may appear to have a normal stretching, however the stretching is computed with respect to N , thus if it spans over a very short time frame, it can still lead to a higher stretching.

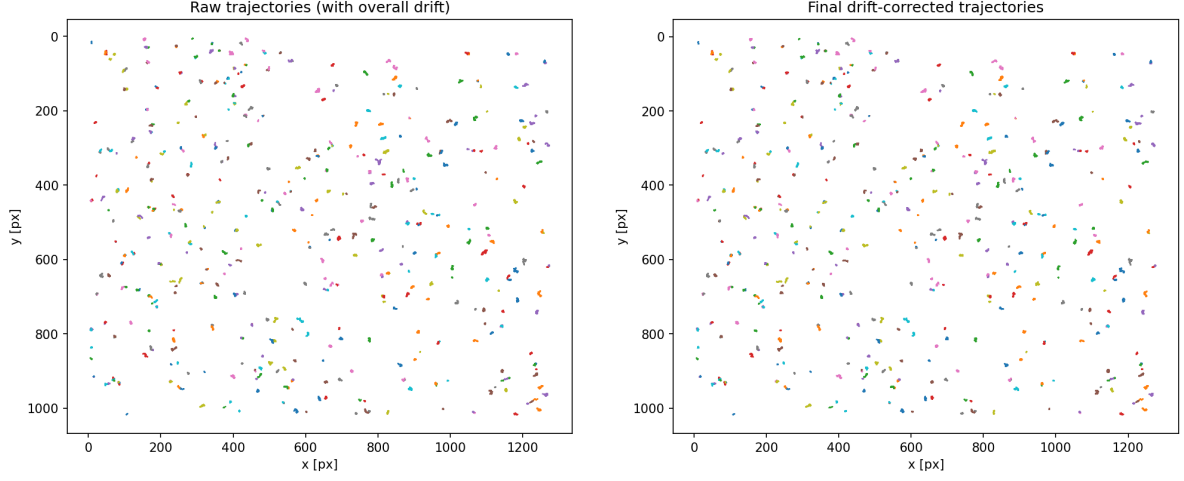


Figure 15: Visual check of drift correction. On the left are shown the trajectories before drift correction, on the right are shown the trajectories after drift correction. In this case, since the drift is very low, we don't see a significant difference between the two sets of trajectories, which also suggests that the drift correction is not introducing any new bias into the dataset.

2.10 Extracting the Diffusion Coefficient

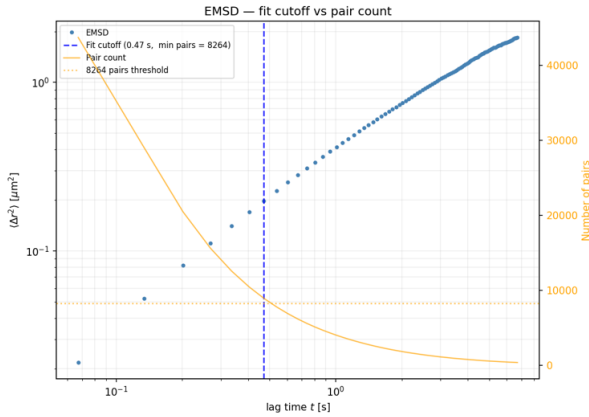


Figure 16: Scatter of every EMSD point as a function of lag time. In yellow is shown the number of independent pair that are used to compute the EMSD. The yellow-dotted horizontal line indicate the minimum number of independent pairs set by our relative error threshold, which gives us the cutoff lag time $\tau_{\max}$ as the blue-dotted vertical line.

We now have a set of trajectories that are cleaned from any significant bias, including residual drift, and we can proceed to the actual analysis of the trajectories to extract the diffusion coefficient. Our population should now be representative of the actual Brownian motion of the particles in the system, and we can use the `imsd` function implemented in `trackpy` to compute individual MSD curves for each trajectory in logarithmic scale to ensure a good visualization of the data across several orders of magnitude in time. As we can see from Fig. 17, MSD curves only show a linear regime at very short lag times ($\tau \approx 1$ second), which is consistent with the expected behavior of Brownian motion. At longer lag times, MSD curves display this "opened tail" behav-

ior. Multiple well-documented sources of bias come into play. First, the finite size of the field of view, which can lead to an underestimation of the MSD at long lag times since particles that move out of the field of view (or the focal plane) are not tracked anymore [11]. Statistical issues arise as well due to the finite length of the trajectories, which can lead to a higher uncertainty in the MSD at long lag times since there are fewer data points available for those lag times [12][13]. We can visualise this effect on Fig. 16. Finally, temperature fluctuations in the system can lead to variations in the diffusion coefficient over time, and thus affect the MSD at longer lag times. To mitigate those issues, we have to set a cutoff lag time $\tau_{\max}$ beyond which we do not consider MSD for fitting anymore. Rather than a visual check, we use the result derived by Qian et al. [13] giving the relative error of the MSD as a function of the number of independent measurements K :

$$\frac{\sigma}{\langle \Delta r^2(\tau) \rangle} \propto \sqrt{\frac{1}{K}} \quad (10)$$

We can extract the number of independent measurements for each lag time from `trackpy` itself, and set a threshold based on the relative error we are willing to accept by inverting Eq. (10) as follows :

$$K(\tau) \propto \frac{1}{x^2} \quad (11)$$

where x is the relative error threshold we set. From here we can set $\tau_{\max}$ as the maximum lag time that satisfies this condition. The value we set for x greatly depends on the quality of the dataset, and changes from one video to another. Setting x too low means that the minimum K required for the fit is too high, which can lead to a very small $\tau_{\max}$, and thus the algorithm is not able to fit the MSD curve properly. Setting x too high means that the minimum K required for the fit is very low, which can lead to an excessively large $\tau_{\max}$, going way beyond the linear regime. In practice, it is a matter of finding the right balance through trial and error, but this

method allows us to set a cutoff lag time that is based on the actual quality of the dataset and the uncertainty in the MSD measurements, rather than relying on arbitrary values or visually based choice. Of course, we can still use a visual check once $\tau_{\max}$ is set to ensure that it is consistent with the expected linear regime of the MSD curve, as shown in Fig. 17.

To derive the diffusion coefficient, we will use the ensemble-averaged MSD (EMSD) across all trajectories rather than individual MSDs, so that we can statistically reduce the impact of most sources of error. We then fit the linear regime of the EMSD curve with the formula provided by Savin and Doyle [12] :

$$\langle \Delta r^2(\tau) \rangle = 4D(\tau - \sigma_{\text{CAM}}/3) + 4\epsilon^2 \quad (12)$$

Here, $\langle \Delta r^2(\tau) \rangle$ is the "observed" MSD, subject to the bias of the localization error and dynamic error. The dynamic error $-\sigma_{\text{CAM}}/3$ accounts for the bias introduced by the finite exposure time of the camera σ_{CAM} , which leads to an underestimation of the MSD at short lag times. The camera doesn't capture the instantaneous position of the particle, but rather an average position over the exposure time, which can lead to a blur effect that reduces the apparent displacement of the particle. The localization error $4\epsilon^2$ accounts for the bias introduced by the uncertainty in the localization of the particle by the camera. Even a fixed particle would still show some apparent flickering due to pixel noise, lighting fluctuations and digital roundings, which can lead to an overestimation of the MSD, especially at short lag times. By fitting the EMSD curve with this formula, we can extract the actual diffusion coefficient D while accounting for these sources of bias, which allows us to obtain a more accurate measurement of the diffusion coefficient from our dataset. This last source of bias can dominate the MSD on the very first lag times. The dynamic error is known since we know the exposure time of our camera, but to get a value for the localization error, as mentioned previously when computing the static threshold, researchers usually compute the STD of a reference population of fixed particles. Our method of using the static particles in our dataset as a reference is not perfect, since those particles may not be perfectly static. It was however a good option since we wanted to get rid of near-static particles as well, but here it can lead to an incorrect estimate of the localization error. To deal with this issue, we decided to let the algorithm fit the localization error as a free parameter, giving it only an order of magnitude based on the static threshold discussed previously so that it can converge properly. To ensure that the fit doesn't suffer too much from this at short lag times, we decided to set a lower bound to our cutoff lag time. Using Savin and Doyle's formula (Eq. (12)), we can look at the ratio δ between the error term and the actual diffusion term :

$$\delta = \frac{4D(\epsilon^2/D - \sigma/3)}{4D\tau} \quad (13)$$

From which we can derive the lower bound $\tau_{\min}$ for the cutoff lag time as a function of the maximum acceptable

ratio $\delta_{\max}$:

$$\tau_{\min} = \frac{|\epsilon^2/D - \sigma/3|}{\delta_{\max}} \quad (14)$$

We decided to set $\delta_{\max}$ to 0.1, which means that we want the error term to be at most 10% of the diffusion term at the minimum cutoff lag time. This allows us to ensure that the fit is not dominated by the localization error at short lag times, while still allowing us to use as much of the linear regime of the EMSD curve as possible for fitting, which can lead to a more accurate estimation of the diffusion coefficient. However this method requires a test run since we need an estimation of both the diffusion coefficient and the localization error to compute the lower bound. Once this value is set, we can compute one last time with both cutoffs to get the value of the diffusion coefficient derived from the EMSD curve.

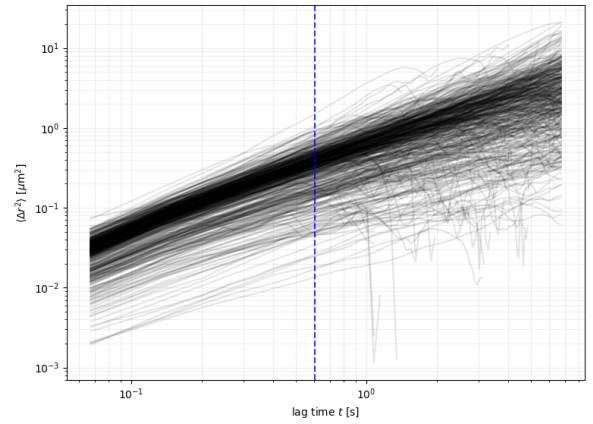


Figure 17: Individual MSD curves for each trajectory in logarithmic scale. The linear regime at very short lag times is consistent with the expected behavior of Brownian motion. At longer lag times, various sources of bias can affect the MSD, as discussed in the text. The blue-dotted line indicates the cut off lag time used for fitting the MSD, so that only the linear regime is considered.

The lag time window $\Delta\tau$ of the linear regime as determined by the two bounds $\tau_{\min}$ and $\tau_{\max}$, varies across videos depending on the quality of the dataset. However the order of magnitude is constant across our whole dataset, around $\Delta\tau = \mathcal{O}(10^{-1})$ s. To gather as much data as possible for the fit and to check for potential variation across the whole video, we compute the EMSD on multiple different and independent sequences per video $\Delta\tau_i$. In order to ensure that those sequences are independent, and that each window has enough data points to be statistically relevant, we set the length of each window to be $\Delta\tau_i = 5\tau_{\max}$, which means that we are using a window that is 5 times larger than the maximum lag time used for fitting the MSD, which should ensure that we have enough data points for the fit while still ensuring enough windows to get a good estimation of the diffusion coefficient. We then use the same cutoff $\tau_{\max}$ and $\tau_{\min}$ for each window, and average the resulting diffusion coefficients across windows to get a final estimation of the diffusion coefficient for each video. This

method allows us to still get a proper estimation of the diffusion coefficient by averaging across multiple independent sequences, while ensuring that we are using the linear regime of the MSD curve for fitting, and more importantly check for variations across the same videos. This idea is driven mostly by the fact that our system is not isolated, which means an ideal thermal equilibrium cannot be reached, hence we want to be able to identify potential temperature variation across a same video, which could lead to an eventual evolution of the diffusion coefficient over time. We acknowledge that this method exposes us to a weaker estimation of D , since by doing so we reduce the amount of data points for each fit, which is why we compare in the results section the full-video D to the windowed mean to ensure that we do not end up with abnormal values. Finally, to ensure that each window is also statistically relevant in terms of the number of trajectories it contains, we set a minimum threshold as $n_{w, \min} = 10$ trajectories per window. We based this threshold on the global $n_{\min} = 15$ frames threshold we set in Section 2.4. We don't expect as much trajectories in each window as we have in the whole video, but we still require a decent amount to get a good estimation of the EMSD curve, and thus the diffusion coefficient. Another advantage of this method, is that we expect the localization error ϵ to be relatively constant across windows, since it is mostly determined by the quality of the video. Hence, we use the ϵ derived from the EMSD of the whole video for the fit of all the other windows, which allows us to reduce the number of free parameters in the fit, and thus get a more robust estimation of D .

3 Results

The complete report of our measurements and the parameters used for each video is provided in the appendix (Section C), but here we will discuss the general trends observed in our results, and how they compare to the expected behavior of Brownian motion. First, in Table 1 we compare the difference between the windowed mean and the full video computation of D . From this we

Table 1: Comparison between full video computation and windowed mean.

Dataset	D (Full)	D (Mean)	Rel. Error
R372	0.0891	0.0889	0.22%
R409	0.1046	0.1021	2.39%
R458	0.1089	0.1085	0.37%
R496	0.1219	0.1214	0.41%
R554	0.1265	0.1252	1.03%
R609	0.1096	0.1127	2.82%
R670	0.1173	0.1196	1.96%
R907	0.1283	0.1280	0.23%

can see that the relative error between the two methods is relatively low, which suggests that the windowing method is effective despite the reduced number of data points for each fit, to provide both a good estimation

of the diffusion coefficient and a check for its stability across the video. We can now confidently check the stability knowing our D values from the windowing method are reliable compared to the classic full video computation. We see with the example of R609 results shown in Fig. 18 that the diffusion coefficient derived from each window of the same video is relatively stable across windows, despite a few peaks that are absorbed in the mean value. We can assert this stability by computing D 's coefficient of variation across all videos, which is simply computed as the standard deviation divided by the mean (Fig. 22). From this figure we can see that except for a peak corresponding to R670, every video sits under a 5% variation threshold, which indicates relatively stable data for every D_i . We do note an increasing trend of the coefficient of variation as temperature decreases. The number of trajectories per window is also relatively stable. However there are a few examples (R496, Fig. 19) where the algorithm could not compute D for some windows. For these windows, the cutoff $\tau_{\max}$ was too low to allow for enough data points to be included in the fit. Since we prioritize quality over quantity, we decided to simply discard these windows rather than setting a higher cutoff that would allow for computation beyond the reliable linear regime.

Once the mean diffusion coefficient is computed for each video, we can plot it as a function of the temperature and the viscosity of the medium (Fig. 20) to check if it follows the expected behavior given by the Stokes-Einstein relation (Eq. (1)). We can see that our results are in good agreement with the Stokes-Einstein relation; D increases with temperature following a linear trend. Our linear fit (black-dotted line) is slightly below the theoretical prediction (green points represent theoretical values at the temperature of each video). The red line, fit to the corrected theoretical data (red triangle), is the theoretical prediction taking into account our measured temperature correction, this will be discussed further in the next section as well. Using Eq. (1), and knowing D , we can now compute the experimentally measured radius $r_{\exp}$ of our particles, and compare it to the manufacturer's radius. We find that our measured radius is in good agreement with the manufacturer's radius of $2.965 \mu\text{m}$, with a value of :

$$r_{\exp} = 3.078 \pm 0.001 \mu\text{m} \quad (15)$$

We can also compute the relative error of our measurement compared to the manufacturer's radius :

$$\epsilon_{\text{rel}} = \frac{|r_{\exp} - r|}{r} \approx 3.8\% \quad (16)$$

We can also verify the characteristic Brownian behaviour of our particles by computing the anomalous exponent α . According to Savin and Doyle[12], the apparent diffusive exponent $\hat{\alpha}$ measured is biased by the localization error and the dynamic error, and can be expressed as a function of the actual diffusive exponent α as follow :

$$\hat{\alpha} = [1 + \epsilon^2/(D\tau) - \sigma_{\text{CAM}}/(3\tau)]^{-1} \quad (17)$$

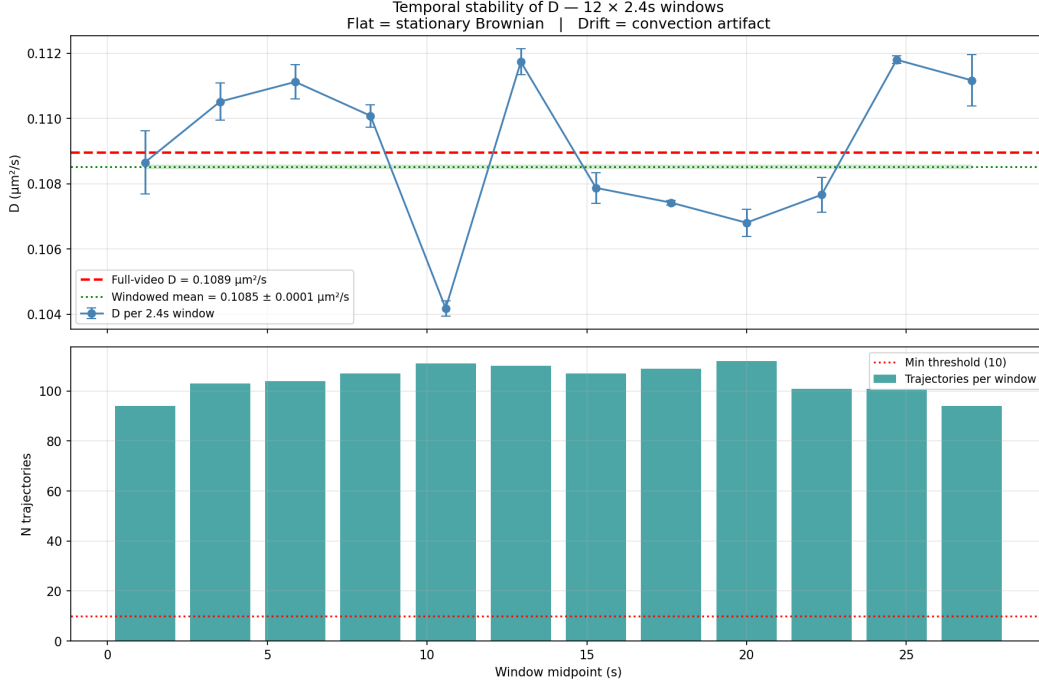


Figure 18: Top : Diffusion coefficient derived from each window of the same video. The green-dotted line indicates the mean value across windows, kept as the final D estimation for this temperature. The red-dotted line indicates the D from the whole-video computation, before windowing. As we can see, the mean value is close to the full-video one. Bottom : Number of trajectories in each window. We can see that the number of trajectories is relatively stable across windows, which suggests that our windowing method is effective in ensuring that each window has enough data points for the fit, and that the population of trajectories is relatively stable across the video. The red-dotted line is our $n_{w, \min}$ threshold.

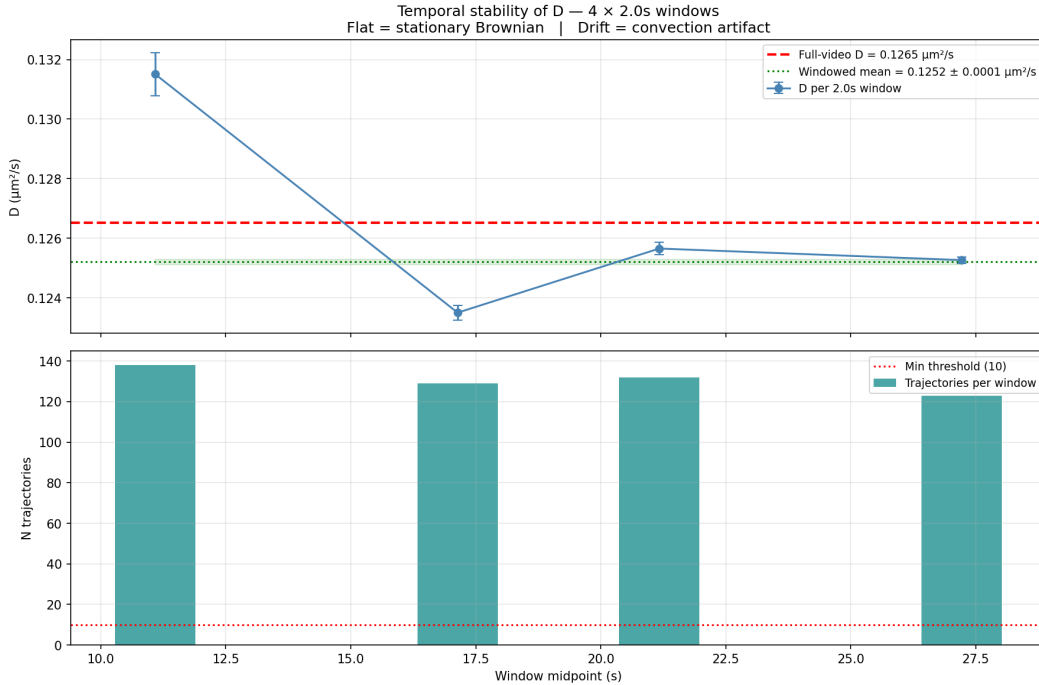


Figure 19: Diffusion coefficient derived from each window of the same video, and histogram of the number of trajectories per window. Just like in Fig. 18, but here is a case where the algorithm struggles to compute D for some windows. However with only 4 data point, the mean value is still relatively stable despite one of the point being a peak, and the relative error between the value derived from the whole video and the mean value across windows is still around 1%, which is a good agreement between the mean method and the classic full video computation.

To easily compute it, we use the D derived from each MSD (hence effectively subtracting both error terms since they are included in the formula from which we derive D) of the first window of each video, and fit it to a power law, to get α . Then taking the mean value of the "corrected" α across every video we get :

$$\alpha = 1.001 \pm 0.007 \quad (18)$$

$$\epsilon_{\text{rel}} \approx 0.001\% \quad (19)$$

This value is in good agreement, and consistent with the expected value of 1 for Brownian motion, which falls in the confidence interval of our measurement. This is a good self-consistency confirmation that our measurement is indeed capturing the Brownian motion of the particles, and that our filters and corrections are effective in mitigating the various sources of bias that can affect the measurement of the diffusion coefficient.

As explained in Section 2.6, we can also compare the static threshold $\hat{\epsilon}_{\text{static}}$ derived from the GMM fit to the localization error ϵ derived from the EMSD fit. Considering the different methods used to derive them and the fact that $\hat{\epsilon}_{\text{static}}$ is meant to capture all static and near-static particles, which can include particles with a slightly higher $\bar{\sigma}_{\text{pos}}$ than the actual localization noise, we cannot expect a perfect agreement. We compare them visually using a scatter plot of $\hat{\epsilon}_{\text{static}}$ as a function of ϵ (Fig. 21). From this plot we gather that there is a good correlation between the two quantities, with most points close to each other. We note that the apparent cluster is above the identity line, which indicates that $\hat{\epsilon}_{\text{static}}$ is generally higher than ϵ , which is consistent with our expectation. This further confirms the consistency of our methods and the reliability of our results. We do note however that we have two dataset, R907 and R496 that seem to be outliers in this plot.

As for the results of our filtering pipeline in terms of usable data, we can finally quantify its average effect across videos by comparing the number of features at the beginning with the number of features at the end of the pipeline. Starting with an average of $f_{\text{start}} = 4834 \pm 420$ features per video, the algorithm narrows it down to an average of $f_{\text{end}} = 403 \pm 150$ features per video, which means that we are keeping on average only 8.34% of the initial features. This is a significant reduction in the number of features, but it is necessary to ensure that we are only keeping high-quality trajectories that are representative of the actual Brownian motion of the particles in the system, and that we are effectively mitigating the various sources of bias that can affect the measurement of the diffusion coefficient.

4 Discussion

The analysis of the temporal stability of D over each video has proven to be a useful tool to confirm the consistency of our temperature across the specific video. There is no general trend of increasing or decreasing D across windows, which suggests that by the time we start tracking, the system has already reached a sufficient thermal equilibrium to ensure stable temperature. However

we observe some smaller and apparently random fluctuations. We noted in the previous section that these fluctuations seem to be following an increasing trend as the temperature decreases. This apparent randomness tends to point towards a signal-to-noise ratio issue, which would be consistent with the decreasing temperature, as D gets smaller but the localisation and dynamic errors remain constant. To confirm this hypothesis, we plotted the signal-to-noise ratio (SNR) of our measurement per video, which we define as the ratio between the diffusion term and the two error terms in the EMSD formula (Eq. (12))(Fig. 23). We do not see any particular trend in the localisation noise evolution, which is consistent with the results in Section C. The dynamic noise decreases as the temperature decreases, which is consistent with the fact that the diffusion term gets smaller. When looking at the SNR, on the left, we can clearly see a peak for the coldest dataset, however this is not a trend as we see in Fig. 22, which suggests that the SNR is not the factor dominating the stability of D across windows. However it is worth noting that our ϵ used for the SNR computation is derived from the first window of each video. So even though ϵ should not fluctuate, this approximation may be distorting our SNR computation. An average of ϵ across windows may give us a better insight, but it would mean fitting two parameters for each window instead of one, hence trading accuracy on the SNR for accuracy on D . We do note that the dynamic error term dominates the static error (hence why our SNR is negative), which means that the finite exposure time of the camera is a significant source of bias in our measurement, even at the lowest temperature we are measuring. This underlines how crucial it is to take this bias into account when fitting the EMSD curve, especially when recording with longer exposure times. The dominating factor in the stability comes from looking at the number of windows computed for each video. As shown in Table 2, some videos have a very low number of windows, which means that the mean value is computed on a very small number of points, and thus the coefficient of variation decreases. This is especially the case for R409, where only 2 windows were computed, which can explain the lowest coefficient of variation observed for this video. This explanation however does not hold for R609 and R372, which have a relatively high number of windows but a lower coefficient of variation. With no clear explanation about these two, we assume they are just particularly stable videos. However despite that clear trend, we still acknowledge that the stability across D_i is great, with only one value above the 5% threshold.

The amount of windows computed for each video however underlines a different problem which is the necessity of a good quality dataset with enough independent pairs to compute the EMSD curve and fit it properly. The target relative error was adjusted per video to address this issue and try to find the optimal window in which to fit each time, but for some videos, it appears that even with an apparently proper video to the human eye, the EMSD algorithm struggles to compute the result over the linear regime. This seems to be a limitation im-

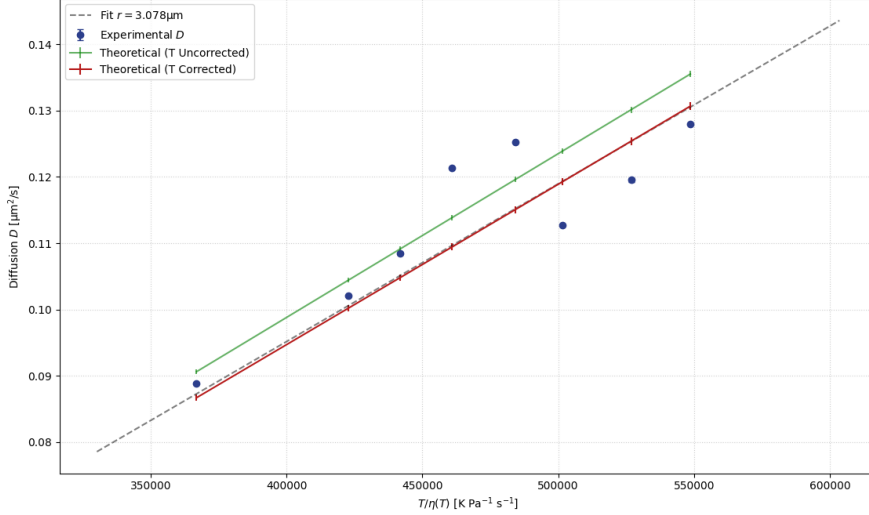


Figure 20: Plot of measured diffusion coefficients D (blue dots) as a function of $T/\eta(T)$, with the linear fit (black-dotted line). The green line is the theoretical value computed at the measured temperature of each video. The red line is the theoretical value computed using the corrected temperature, taking into account our measured temperature offset ΔT . The linear fit has a slightly smaller slope than the theoretical prediction, and the vertical position of the fit, directly related to the temperature is consistent with the corrected theoretical prediction as well, which seems to confirm our offset measurement. No error bars are visible since through our computation method the resulting error on D are extremely small. Theoretical value are computed using the Stokes-Einstein relation, with the radius of the particles set to the manufacturer's radius and the temperature error taken into account for both T and ηT .

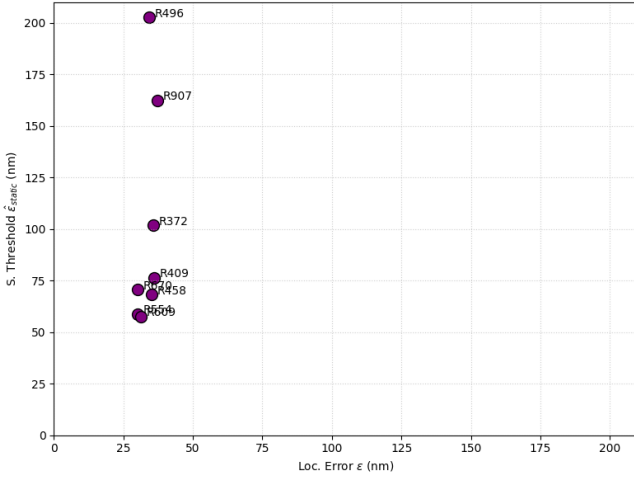


Figure 21: Scatter plot of the static threshold $\hat{\epsilon}_{\text{static}}$ derived from the GMM fit as a function of the localization error ϵ derived from the EMSD fit.

posed by the experimental setup, however we managed to find proper results for our whole dataset, which is a good confirmation of the robustness of our method and the importance of having enough statistically independent samples to get a good estimation of the diffusion coefficient. This issue would have been minimized with significantly longer videos, but we were limited by the size of the files we could record on the computer of the microscope if we were to avoid longer wait time and po-

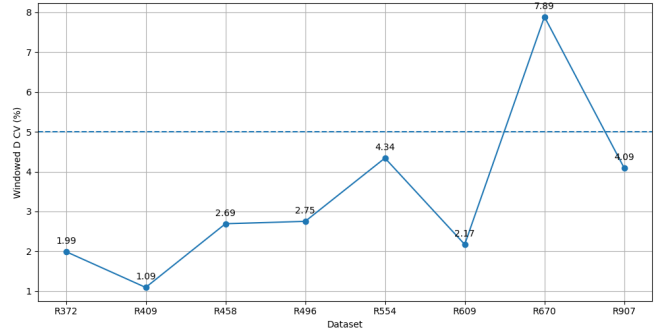


Figure 22: Temporal stability of diffusion coefficients represented by coefficient of variation of D expressed in percentage. The dashed line indicates a stability threshold of 5% as comparison. We see a clear trend of increasing as the temperature decreases.

tential crashes. This issue could easily be addressed in future experiments, and would be a good way to confirm our hypothesis about the fluctuations of D_i and their trend.

The theoretical prediction of the Stokes-Einstein relation is in good agreement with our results. Since the temperature remained constant, the slope of Fig. 20 depends solely on the particle radius, r . Our analysis yields a radius 3.8% larger than the value provided by the manufacturer. Considering the fact that the manufacturer's radius is given without tolerance, and given the statistical nature of our process on a large dataset, it

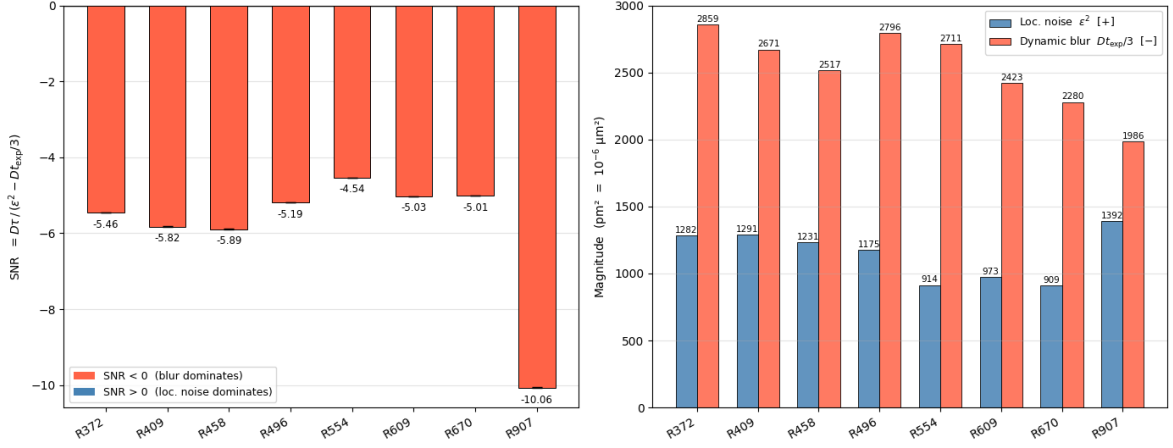


Figure 23: Left : Signal-to-noise ratio (SNR) of the measurement per video, negative since the dynamic error term (blur due to the camera finite exposure time) dominates over the diffusion term. Right : Comparison between the blur term and the diffusion term for each video.

Table 2: Summary of Windows Computed per Dataset

Dataset	Windows computed
R372	11
R409	2
R458	3
R496	4
R554	11
R609	12
R670	12
R907	9

is not expected to find the manufacturer’s value within the confidence interval of our measurement. The relative error is by itself a great confirmation of our results. However the temperature offset Δ measured at room temperature also seems to be consistent with our experimental data, as shown in the plot, where the linear fit and the theoretical data almost overlap. It also indicates that the temperature offset seems to be constant in this range of temperature of the resistance. We can look at the individual errors of our data points compared to this corrected theoretical prediction in Table 3. From that

Table 3: Experimental vs. Theoretical Diffusion Coefficients

ID	Exp. D ($\mu m^2/s$)	Theo. D Corr ($\mu m^2/s$)	Error (%)
R372	0.1280 ± 0.0000	0.1307 ± 0.0006	2.06
R409	0.1196 ± 0.0003	0.1254 ± 0.0006	4.65
R458	0.1127 ± 0.0001	0.1193 ± 0.0006	5.48
R496	0.1252 ± 0.0001	0.1151 ± 0.0005	8.81
R554	0.1214 ± 0.0001	0.1095 ± 0.0005	10.91
R609	0.1085 ± 0.0001	0.1048 ± 0.0005	3.55
R670	0.1021 ± 0.0001	0.1002 ± 0.0005	1.87
R907	0.0889 ± 0.0001	0.0867 ± 0.0005	2.53

table we can see that the relative error except for R496 and R554 is under the reasonable range of 6%, which is

once again a good sign. However there is no clear sign pointing towards a statistical unreliability of these two datasets. Looking at the GMM static threshold $\hat{\epsilon}_{static}$ derived for R496, we can see that it is relatively high compared to the other videos, which would be consistent with a D abnormally high if the filter got rid of slow but non static particles, effectively dragging D up. It seems that in this case, the GMM failed to capture the actual distribution of static and near-static particles. We confirm this by looking at their distribution of $\bar{\sigma}_{pos}$ (Fig. 24), which shows three populations of particles, when the GMM is looking for only two, with a clear peak of low but not static particles. This is a good example of the limitations of the GMM method, which can struggle to capture complex distributions with multiple populations, especially when the populations are not well separated, or when one of them is relatively small compared to the others, even in log-space. This disturbance is effectively shifting the static threshold to a higher value, which means the filter is getting rid of slower particles that would be included in the Brownian population, hence giving the D a bias towards a higher value. However since we want to make this pipeline as human-independent as possible, we decided to keep this value for the static filter as a clear sign of the limitations of this method. R554 on the other hand does not show any particular issue with the GMM fit, and the static threshold is relatively similar to the other videos. With no sign of statistical unreliability, and parameters within the expected range, we do not have a clear explanation for the high relative error of this video, but we can assume that it may be a statistical fluctuation, or that there is some other source of bias that we have not taken into account in our analysis, affecting this specific video. However, even with this high relative error, the overall trend of D as a function of temperature is consistent across all videos, which is a good confirmation of the reliability of our results and our method, which is able to capture the expected behavior of Brownian motion and the Stokes-Einstein relation, even in the presence of some outliers or fluctuations in the data. Once again

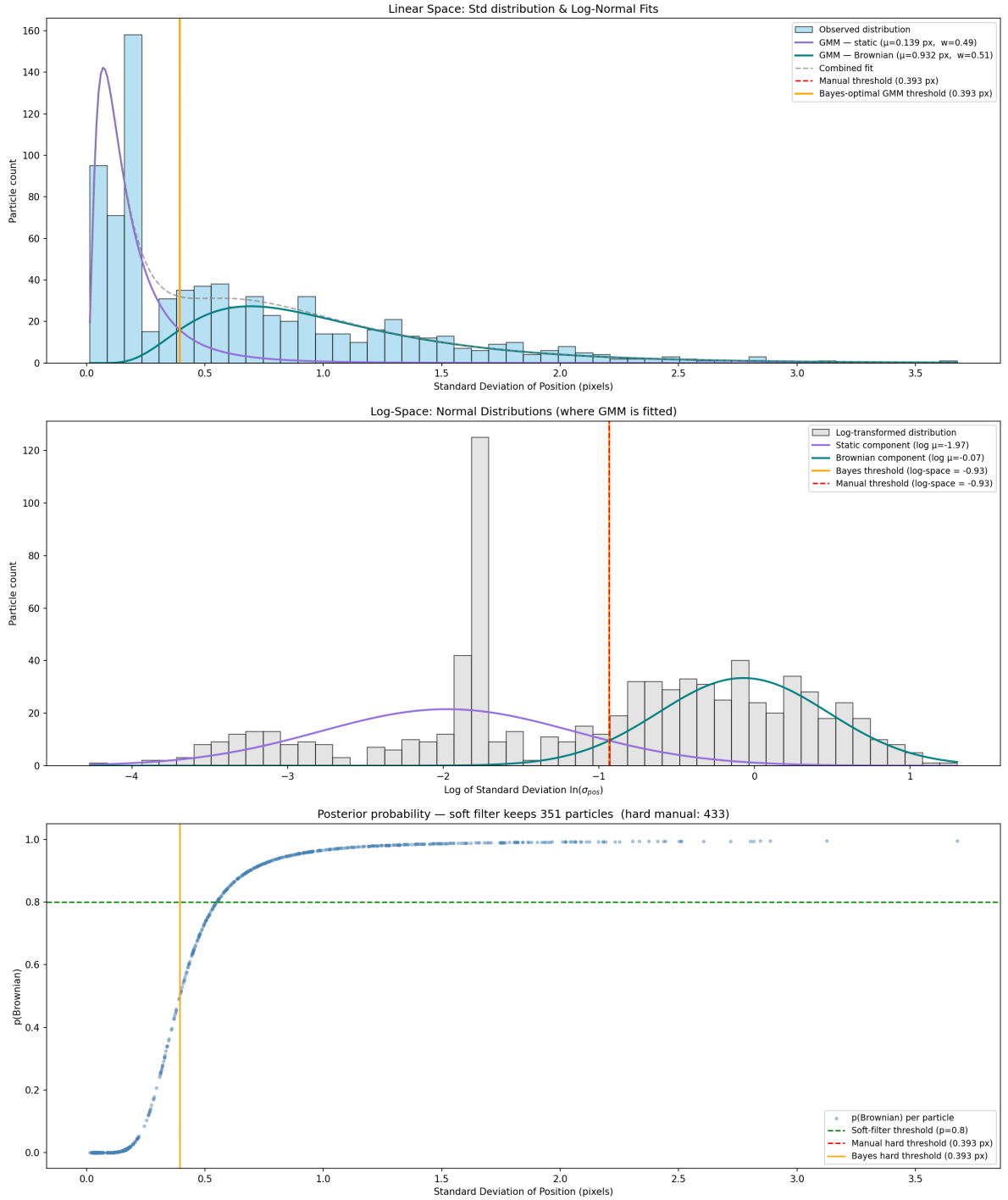


Figure 24: GMM analysis of the $\bar{\sigma}_{pos}$ distribution for R496. The GMM is trying to fit two Gaussian distributions (purple and green) to the data, but there are clearly three distinct populations of particles, with a clear peak around $0.2 - 0.25 \mu\text{m}$ that is not captured by the GMM. This suggests that the GMM is not able to properly capture the distribution of static and near-static particles in this case, which could explain the abnormally high diffusion coefficient derived from this video.

it underlines the importance of having a good quality dataset with enough independent samples to get a reliable estimation of the diffusion coefficient, and the limitations of our method when dealing with complex distributions of particles or potential sources of bias that are not accounted for. The diffusive exponent that we extracted from our dataset is a solid verification of our filtering pipeline. We find a coefficient 0.001% higher than the expected value, which is a very good result considering the various sources of bias that can affect our experimental apparatus. One can consider this result biased by the fact that we based one of our filter parameters on the expected value of α , but it is worth noting that the value of α is derived from the same MSD curve as D , so even if we are using the expected value of α to set a cutoff for the fit, we are not directly using it to compute D . Obviously, we got rid of sub-diffusive particles, but they are to be expected considering our apparatus, and we have to filter them to capture the actual Brownian population, which we can now confidently certify is behaving according to Brownian motion. We can also note that the correction proposed by Savin and Doyle to take into account the various sources of bias in the EMSD curve gave excellent results considering our relative error, this is a good confirmation of the robustness of their method as well. We can compare the raw and the corrected α from Section C in the following table (Table 4) : We do note that the raw α values are all

Table 4: Comparison of raw and corrected α values

Dataset	α raw	α corrected
R372	1.045	1.000 ± 0.001
R409	1.047	1.001 ± 0.003
R458	1.046	1.001 ± 0.001
R496	1.054	1.001 ± 0.001
R554	1.054	1.001 ± 0.001
R609	1.051	1.000 ± 0.001
R670	1.057	1.002 ± 0.003
R907	1.039	1.002 ± 0.005

above 1, which looking at Eq. (17) is consistent with our previous observation of a negative SNR, since the error terms are dominated by the dynamic error that tends to drag $\hat{\alpha}$ up.

We have also seen that the static threshold derived from the GMM fit is overall higher than the localization error derived from the EMSD fit. From the two outliers that we noted in the scatter plot of Fig. 21 as R907 and R496, R496 has already been accounted for in the discussion of the high relative error of this video. R907’s GMM static threshold is relatively high as well, once again due to the fact that the GMM is struggling to capture the actual distribution of static and near-static particles, especially since this is the lowest temperature video, which means that Brownian particles are slower and thus more likely to be included in the static population. Considering our results, the GMM probably overestimated the static threshold, and got rid of a few slower particles, but the core of the Brownian population is still included in the fit, giving us the third best error relative to the

theoretical prediction.

Finally, we can look at the overall efficiency of our pipeline. We are keeping on average only 8.34% of the initial features, which is a significant reduction, but it is necessary to ensure that we are only keeping high-quality trajectories that are representative of the actual Brownian motion of the particles in the system, and that we are effectively mitigating the various sources of bias that can affect the measurement of the diffusion coefficient. Once again, this requires a good quality dataset with enough independent samples to obtain a reliable statistical estimate. However, even with this significant reduction in the number of features, we are still able to capture the expected behavior of Brownian motion and the Stokes-Einstein relation, which is a good confirmation of the robustness of our method and its ability to extract meaningful information from complex datasets, even in the presence of noise and potential sources of bias. One of the core ideas behind this pipeline was to minimize as much as possible human-dependent bias, by setting clear and objective criteria for each filter, such as the GMM method for the static filter, or the α cutoff for sub-diffusive particles. This approach has proven to be effective in ensuring the consistency and reliability of our results, given that we have enough independent samples. This method would statistically improve with longer videos, or more videos at different temperatures. However we have seen the limitations of our method when dealing with complex datasets, where despite the efficiency of algorithms, human judgement is still required to ensure that the results are physically sound. It is also important to note that despite constant efforts to minimize human bias, there are still some parameters in our pipeline that are set through trial and error to find which combination successfully targets the intended population, such as the α cutoff for sub-diffusive particles, and the stretching factor used to filter flow-like trajectories. Despite that, we have tried to be as consistent as possible and keep these parameters fixed throughout the whole dataset, to ensure that we are not tuning them to fit our expected results, but rather to find a good compromise that can be applied across the whole dataset. An honest example of this limitation is the last dataset, R907, where we identified a significant sub-population that was not Brownian, and that we had to filter out by tweaking the stretching factor (Fig. 25). Effectively, the GMM algorithm could not properly compute this population and we had to manually move the threshold. We acknowledge that this is an significant source of bias, especially considering our intended approach, this is why we kept this dataset in the final results as a clear example. This once again highlights the fact that erratic behavior can still be present in the data after our filtering pipelines, and that it is crucial to carefully analyze the results and make sure they are physically sound.

5 Conclusion

In this study, we successfully established and validated a robust experimental and analytical framework for the

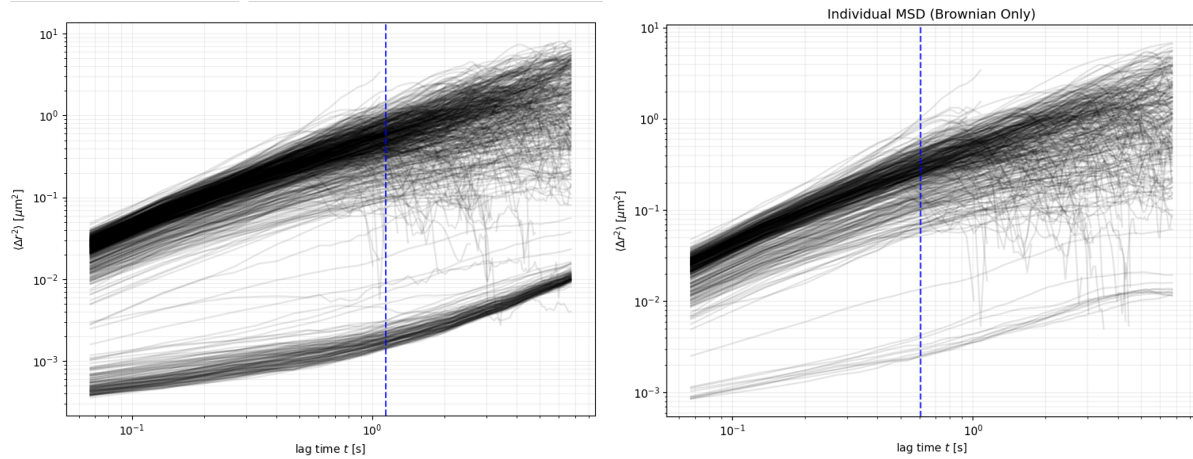


Figure 25: Left : ISF plot of the uncorrected R907 video, where we can clearly see another population undergoing sub-diffusion. Right : ISF plot of the corrected R907 video, where we applied a more stringent filter to get rid of the sub-diffusive population.

quantitative analysis of Brownian motion using 2D video microscopy. By investigating the diffusion of $5.93\text{ }\mu\text{m}$ polyester beads across a temperature range of $28.56\text{ }^{\circ}\text{C}$ to $46.30\text{ }^{\circ}\text{C}$, we confirmed a clear linear correlation between the diffusion coefficient D and temperature T . This relationship was found to be highly consistent with the theoretical predictions of the Stokes-Einstein relation. A central achievement of this work was the implementation of a multi-step analytical pipeline designed to minimize user-dependent bias and ensure high reproducibility. By utilizing the Crocker-Grier algorithm through `trackpy` and introducing advanced filtering layers such as Gaussian Mixture Model (GMM) discrimination for static particles and anomalous exponent (α) filtering for encumbered trajectories, we were able to isolate a high-fidelity dataset of particles undergoing true Brownian motion. The measured radius of the beads closely matched the manufacturer's specifications, further validating the accuracy of our calibration and tracking methodologies. While our "bias-free" approach proved effective even in our experimental environment, we identified certain limitations, particularly the challenges in separating populations in certain datasets where the GMM-based discrimination becomes less reliable. Nevertheless, the protocol developed provides a practical and consistent methodology for studying colloidal systems. Future improvements could focus on identifying precisely what behavior is causing algorithms such as the GMM to struggle, and how to adapt them better to capture the intended distribution of particles. This could be done by testing the algorithm on synthetic data with known distributions, or by implementing more complex models that can capture multiple populations of particles. Additionally, increasing the length of the videos or the number of videos at different temperatures could provide more independent samples and improve the statistical reliability of our results. Overall, this work also highlights the importance of careful experimental design and rigorous data analysis in the study of Brownian motion, and provides us with a foundation for future investigations into the dynamics

of colloidal systems and other complex fluids.

References

- [1] John C. Crocker and David G. Grier. "Methods of Digital Video Microscopy for Colloidal Studies". In: *Journal of Colloid and Interface Science* 179.1 (1996), pp. 298–310. ISSN: 0021-9797. DOI: <https://doi.org/10.1006/jcis.1996.0217>. URL: <https://www.sciencedirect.com/science/article/pii/S0021979796902179>.
- [2] Dan Allan et al. *trackpy: Trackpy v0.7.0*. Version v0.7.0. Oct. 2025. DOI: [10.5281/zenodo.591577](https://doi.org/10.5281/zenodo.591577). URL: <https://github.com/soft-matter/trackpy>.
- [3] Caroline A. Schneider, Wayne S. Rasband, and Kevin W. Eliceiri. "NIH Image to ImageJ: 25 years of image analysis". In: *Nature methods* 9.7 (2012), pp. 671–675. DOI: [10.1038/nmeth.2089](https://doi.org/10.1038/nmeth.2089).
- [4] H. B. Eral, F. Mugele, and M. H. G. Duits. "Colloidal Dynamics Near a Particle-Covered Surface". In: *Langmuir* 27.20 (2011). PMID: 21827156, pp. 12297–12303. DOI: [10.1021/la2024764](https://doi.org/10.1021/la2024764). URL: <https://doi.org/10.1021/la2024764>.
- [5] A.J. Goldman, R.G. Cox, and H. Brenner. "Slow viscous motion of a sphere parallel to a plane wall—I Motion through a quiescent fluid". In: *Chemical Engineering Science* 22.4 (1967), pp. 637–651. ISSN: 0009-2509. DOI: [https://doi.org/10.1016/0009-2509\(67\)80047-2](https://doi.org/10.1016/0009-2509(67)80047-2). URL: <https://www.sciencedirect.com/science/article/pii/0009250967800472>.
- [6] Juan José Gómez Romera. *jgomera/iapws: version v1.5.2*. May 2021. DOI: [10.5281/zenodo.4744318](https://doi.org/10.5281/zenodo.4744318). URL: <https://doi.org/10.5281/zenodo.4744318>.

- [7] Anne Kowalczyk, Claude Oelschlaeger, and Norbert Willenbacher. “Tracking errors in 2D multiple particle tracking microrheology”. In: *Measurement Science and Technology* 26.1 (Dec. 2014), p. 015302. DOI: [10.1088/0957-0233/26/1/015302](https://doi.org/10.1088/0957-0233/26/1/015302). URL: <https://doi.org/10.1088/0957-0233/26/1/015302>.
- [8] X. Michalet. “Mean square displacement analysis of single-particle trajectories with localization error: Brownian motion in an isotropic medium”. In: *Physical Review E* 82.4 (Oct. 2010). PMID: 21230320; PMCID: PMC3055791, p. 041914. DOI: [10.1103/PhysRevE.82.041914](https://link.aps.org/doi/10.1103/PhysRevE.82.041914). URL: <https://link.aps.org/doi/10.1103/PhysRevE.82.041914>.
- [9] C. Schirripa Spagnolo and S. Luin. “Trajectory Analysis in Single-Particle Tracking: From Mean Squared Displacement to Machine Learning Approaches”. In: *International Journal of Molecular Sciences* 25.16 (Aug. 2024), p. 8660. DOI: [10.3390/ijms25168660](https://doi.org/10.3390/ijms25168660).
- [10] M.-L. Jobin et al. “Filamin A organizes γ -aminobutyric acid type B receptors at the plasma membrane”. In: *Nature Communications* 14.1 (Jan. 2023), p. 34. DOI: [10.1038/s41467-022-35708-1](https://doi.org/10.1038/s41467-022-35708-1).
- [11] Anders S Hansen et al. “Robust model-based analysis of single-particle tracking experiments with Spot-On”. In: *eLife* 7 (Jan. 2018). Ed. by David Sherratt, e33125. ISSN: 2050-084X. DOI: [10.7554/eLife.33125](https://doi.org/10.7554/eLife.33125). URL: <https://doi.org/10.7554/eLife.33125>.
- [12] Thierry Savin and Patrick S. Doyle. “Static and Dynamic Errors in Particle Tracking Microrheology”. In: *Biophysical Journal* 88.1 (2005), pp. 623–638. ISSN: 0006-3495. DOI: <https://doi.org/10.1529/biophysj.104.042457>. URL: <https://www.sciencedirect.com/science/article/pii/S0006349505731362>.
- [13] Hong Qian, Michael P. Sheetz, and Elliot L. Elson. “Single particle tracking. Analysis of diffusion and flow in two-dimensional systems”. In: *Biophysical Journal* 60.4 (Oct. 1991), pp. 910–921. DOI: [10.1016/S0006-3495\(91\)82125-7](https://doi.org/10.1016/S0006-3495(91)82125-7).

Appendix A : Derivation of the maximum search range radius formula

In this appendix, we derive the formula for the maximum search range r_s used in Section 2.3. We start by assuming that the particle undergoes 2D Brownian motion, which can be described by a Gaussian distribution of displacements with a mean squared displacement given by Eq. (5). The probability density function (PDF) for the particle to be in position x after a time interval Δt is given by:

$$P(x) = \frac{1}{\sqrt{4\pi D\Delta t}} \exp\left(-\frac{x^2}{4D\Delta t}\right) \quad (\text{A.1})$$

Since x and y are independent, the joint PDF for the 2D displacement is:

$$P(x, y) = P(x)P(y) = \frac{1}{4\pi D\Delta t} \exp\left(-\frac{x^2 + y^2}{4D\Delta t}\right) \quad (\text{A.2})$$

Switching to polar coordinates, where $r = \sqrt{x^2 + y^2}$, we can express the PDF in terms of r only (integrating out the angular component):

$$P(r) = \frac{r}{2D\Delta t} \exp\left(-\frac{r^2}{4D\Delta t}\right) \quad (\text{A.3})$$

Integrating now the PDF from 0 to r_s gives us the cumulative distribution function (CDF), which represents the probability that the particle's displacement is less than or equal to r_s :

$$C(r_s) = \int_0^{r_s} P(r)dr = 1 - \exp\left(-\frac{r_s^2}{4D\Delta t}\right) \quad (\text{A.4})$$

Since we're interested in the probability of a particle's displacement being greater than or equal to r_s (i.e., the probability of missing the particle), we can express this as:

$$1 - C(r_s) = P_{\text{miss}} \quad (\text{A.5})$$

Rearranging this equation gives us:

$$\exp\left(-\frac{r_s^2}{4D\Delta t}\right) = P_{\text{miss}} \quad (\text{A.6})$$

Taking the natural logarithm of both sides, we get:

$$-\frac{r_s^2}{4D\Delta t} = \ln(P_{\text{miss}}) \quad (\text{A.7})$$

Finally, solving for r_s gives us the formula:

$$r_s = \sqrt{-4D\Delta t \ln(P_{\text{miss}})} \quad (\text{A.8})$$

Appendix B : Derivation of the stretching ratio

As seen in Appendix A, the PDF for a 2D Brownian displacement is given by :

$$P(r) = \frac{r}{2D\Delta t} \exp\left(-\frac{r^2}{4D\Delta t}\right) \quad (\text{B.1})$$

Which can be rewritten using the 2D relationship :

$$\sigma^2 = 2D\Delta t \quad (\text{B.2})$$

We can then express the PDF as :

$$P(r) = \frac{r}{\sigma^2} \exp\left(-\frac{r^2}{2\sigma^2}\right) \quad (\text{B.3})$$

The mean displacement can then be calculated as :

$$\langle \Delta r \rangle = \int_0^\infty r \cdot P(r)dr = \int_0^\infty \frac{r^2}{\sigma^2} \exp\left(-\frac{r^2}{2\sigma^2}\right) dr = \sigma \sqrt{\frac{\pi}{2}} \quad (\text{B.4})$$

Similarly, we can compute the end-to-end distance R of a Brownian trajectory after N steps by considering it as one single step with a displacement that follows the same distribution as $P(r)$, but with a variance that is N

times larger, since the displacements are independent and identically distributed. This means that the PDF for the end-to-end distance can be expressed as :

$$P(R) = \frac{R}{N\sigma^2} \exp\left(-\frac{R^2}{2N\sigma^2}\right) \quad (\text{B.5})$$

Therefore $\langle R \rangle$ is given by :

$$\langle R \rangle = \int_0^\infty R \cdot P(R) dR = \int_0^\infty \frac{R^2}{N\sigma^2} \exp\left(-\frac{R^2}{2N\sigma^2}\right) dR = \sqrt{N} \sqrt{\pi D \Delta t} \quad (\text{B.6})$$

The length of the trajectory after N independent steps can be approximated as :

$$\langle L \rangle = N\sigma = N\sqrt{2D\Delta t} \quad (\text{B.7})$$

Finally, the expected stretching ratio can be calculated as :

$$\langle S \rangle = \frac{\langle R \rangle}{\langle L \rangle} = \frac{\sqrt{N} \sqrt{\pi D \Delta t}}{N \sqrt{2D \Delta t}} = \sqrt{\frac{\pi}{2N}} \quad (\text{B.8})$$

This expression of $\langle L \rangle$ could have also been written as $N\langle \Delta r \rangle$, which would have given a slightly higher value of $\langle S \rangle$ of $\sqrt{\pi}/\sqrt{2N}$. We choose here the less lenient expression to set a more strict threshold for our flow-like trajectories filter, since we want to minimize the risk of including those trajectories in our dataset.

Appendix C : Summary of measurement results for every dataset

Table C.1: Detailed analysis report of the selected dataset

Dataset	R372	R409	R458	R496	R554	R609	R670	R907
VIDEO METADATA								
FPS (fps)	14.88	14.88	14.88	14.88	14.88	14.88	14.88	14.88
Duration (s)	29.84	29.77	29.84	29.84	29.84	29.9	29.84	29.77
Frames	444	443	444	444	444	445	443	443
Effective interval (s)	0.0672	0.0672	0.0672	0.0672	0.0672	0.0672	0.06735	0.0672
TRACKING PARAMETERS								
Diameter (px)	13	13	13	13	13	13	13	13
Min mass	1700	1700	1700	1700	1700	1700	1550	2000
Search range (px)	0.8	0.8	0.8	0.8	0.8	0.7	0.7	0.7
Memory (frames)	2	2	2	2	2	2	2	2
Pixel size ($\mu\text{m}/\text{px}$)	0.5156	0.5156	0.5156	0.5156	0.5156	0.5156	0.5156	0.5156
Exposure time (s)	0.067	0.067	0.067	0.067	0.067	0.067	0.067	0.067
FILTERING PARAMETERS								
Stubs min. frames (frames)	15	15	15	15	15	15	15	15
Quality mass range	[1800, 2450]	[1800, 2450]	[1600, 2250]	[1750, 2350]	[1750, 2350]	[1650, 2350]	[1550, 2350]	[2000, 2500]
Quality size range (R_g) (px)	[2.2, 2.55]	[2.2, 2.5]	[2.2, 2.5]	[2.1, 2.45]	[2.1, 2.45]	[2.1, 2.45]	[2.25, 2.5]	[2.25, 2.55]
Static threshold (manual) (px)	0.197	0.148	0.132	0.393	0.114	0.112	0.137	0.555
Static threshold (GMM) (px)	0.1973	0.1484	0.1324	0.393	0.1143	0.112	0.137	0.3151
α minimum	0.75	0.75	0.75	0.75	0.75	0.75	0.75	0.75
Flow K stretch	1.5	1.5	1.5	1.5	1.5	1.5	1.5	1
FILTERING PIPELINE (Before $\rightarrow$ After)								
1. Stubs	4829 $\rightarrow$ 1742	5676 $\rightarrow$ 1917	5213 $\rightarrow$ 1667	4796 $\rightarrow$ 1596	4485 $\rightarrow$ 1414	4438 $\rightarrow$ 1508	4929 $\rightarrow$ 1591	4308 $\rightarrow$ 1617
2. Quality	1742 $\rightarrow$ 1067	1917 $\rightarrow$ 998	1667 $\rightarrow$ 764	1596 $\rightarrow$ 805	1414 $\rightarrow$ 692	1508 $\rightarrow$ 846	1591 $\rightarrow$ 526	1617 $\rightarrow$ 1185
3. Static	1067 $\rightarrow$ 1006	998 $\rightarrow$ 718	764 $\rightarrow$ 464	805 $\rightarrow$ 351	692 $\rightarrow$ 427	846 $\rightarrow$ 561	526 $\rightarrow$ 354	1185 $\rightarrow$ 1118
4. α	1006 $\rightarrow$ 869	718 $\rightarrow$ 597	464 $\rightarrow$ 374	351 $\rightarrow$ 333	427 $\rightarrow$ 367	561 $\rightarrow$ 458	354 $\rightarrow$ 298	1118 $\rightarrow$ 903
5. Flow (final N)	869 $\rightarrow$ 725	597 $\rightarrow$ 514	374 $\rightarrow$ 329	333 $\rightarrow$ 288	367 $\rightarrow$ 317	458 $\rightarrow$ 382	298 $\rightarrow$ 231	903 $\rightarrow$ 442
THERMODYNAMICS								
Resistance (Ω)	3720	4090	4580	4960	5540	6090	6700	9070
Temperature ($^{\circ}\text{C}$)	46.3	44.35	42.04	40.42	38.2	36.32	34.43	28.56
D_{theo} ($\mu\text{m}^2 \text{ s}^{-1}$)	0.1355 ± 0.000463	0.1301 ± 0.000449	0.1239 ± 0.000434	0.1196 ± 0.000423	0.1139 ± 0.000408	0.1091 ± 0.000396	0.1045 ± 0.000384	0.09064 ± 0.000347
MSD FITTING & DIFFUSION								
D_{exp} (WMSD) ($\mu\text{m}^2 \text{ s}^{-1}$)	$0.128 \pm 2.9e-05$	0.1196 ± 0.000276	$0.1127 \pm 9.5e-05$	$0.1252 \pm 8.9e-05$	$0.1214 \pm 8e-05$	$0.1085 \pm 5.6e-05$	0.1021 ± 0.000139	$0.08891 \pm 7.5e-05$
Fitting error (%)	0.23	0.34	0.31	0.31	0.21	0.17	0.2	0.7
Error vs. theory (%)	5.57	8.12	9	4.67	6.6	0.55	2.26	1.91
Loc. ϵ (nm)	35.81	35.93	35.09	34.28	30.24	31.19	30.15	37.31
Static threshold ϵ_{static} (nm)	101.8	76.5	68.3	202.6	58.9	57.7	70.6	162.5
SYSTEM STABILITY								
Windowed D CV (%)	1.99	1.09	2.69	2.75	4.34	2.17	7.89	4.09
Windows computed	11	2	3	4	11	12	12	9
SCALING & CUTOFFS								
Lower cutoff (s)	0.1564	0.1599	0.1572	0.1723	0.1704	0.1624	0.159	0.1061
Upper cutoff (s)	0.54	0.4	0.4	0.4	0.54	0.47	0.4	0.6
α raw	1.045	1.047	1.046	1.054	1.054	1.051	1.057	1.039
α corrected	1.000 ± 0.001	1.001 ± 0.003	1.001 ± 0.001	1.001 ± 0.001	1.001 ± 0.001	1.000 ± 0.001	1.002 ± 0.003	1.002 ± 0.005
SELF-CONSISTENCY								
Adv. search range (px)	0.77	0.74	0.71	0.77	0.75	0.71	0.69	0.64

Appendix D : List of Equipment

Microscope: <https://physique.ens-lyon.fr/sites/default/files/materiel-physique-notices/N000-083.pdf>

Camera: <https://www.emsdiasum.com/docs/technical/datasheet/95102.pdf?srsltid=AfmB0oo82UTZ0J32Y0QauasSdcEqaf3LX7vt-GYMYBT3J9qwmbdg5S6>

Resistances: <https://www.physics.utoronto.ca/apl/ram/GCH25-75-Manual.pdf>

Multimeter: (Older model reference) <https://paraset.nl/sh/product/dmm-itc-996/>

Micropipettes: Model H (20–200 μL & 100–1000 μL) http://www.hwlab.cn/product/micropipette_h_model.html

Generator: https://www.aemc.com/userfiles/files/resources/usermanuals/Power-Supplies/AX501-502-503_EN.pdf